FYP Sem 7 — Complete Viva Preparation Guide

Smart Home HAR for Speech-Impaired & Hearing-Impaired Users 23CSE399 Project Phase 2 · Guide Review 1 | Panel Review 1 Guide: Ms. Nalinadevi K · Dept. of CSE, Amrita Vishwa Vidyapeetham Team: Adhikkesh S K, Adithya Ajay, Amrith B, Gowreesh B, Rishiikesh S K

How to read this document

Every factual claim in this guide carries one of four tags. **Do not blur them in the viva.**

Tag	Meaning
[V] Verified	Read directly out of the repository — a file, a measured number, a line of code. You can defend it and point at the source.
[M] Measured here	I re-measured it from the raw data during this analysis. Reproducible with the command given.
[S] Standard knowledge	Established engineering/physics, not specific to this project. Safe to state as general fact.
[L] Likely rationale	A plausible technical justification that the repository does **not** explicitly state. **Say "our reasoning was…", never "we proved…"**

If you are asked something not covered here, the correct answer is *"I'd have to check the code for that"* — never invent a number.

---

1. Executive Project Overview

1.1 What the project actually is

The project builds a **camera-free, wearable-free sensing layer for a smart home**, using ordinary WiFi radio signals as the sensor. The goal is to recognise what a person is doing — and eventually to recognise sign language — for two user groups: **speech-impaired** users, who *produce* sign language, and **hearing-impaired** users, who need household events *surfaced visually* rather than audibly.

The sensing quantity is **Channel State Information (CSI)**: the complex frequency response of the radio channel between a transmitter and a receiver, measured on each of the 52 usable OFDM subcarriers of an IEEE 802.11a/g frame. When a human body moves between the two antennas, the multipath structure of the room changes, so CSI changes. A CSI time series is therefore a recording of motion — with no image and no audio in it. **[V]**

1.2 The single most important thing to understand before the viva

The repository contains **two physically different systems**, and the review deck presents them as one continuous story. They are not.

	Setup 1 — passive (`USRP_lib/`)	**Setup 2 — active** (`Three_Channel_Experiment/`)
Radios	**One** USRP N210 (receiver only)	**Two** USRP N210 (dedicated TX + RX)
Illumination	A real MacBook ↔ Android-hotspot link	Our own transmitter, `packet_generator.py` at 200 pps
Channel	Parked, channel 11 (2462 MHz)	Hops 1 → 6 → 11, 100 ms dwell (or parks)
CSV header	`Packet,Timestamp_us,SC0_Re,…`	`Packet,Timestamp_us,` **`Channel`** `,SC0_Re,…`
Delivered CSI rate	**51.5 – 81.1 Hz**	**1.96 Hz** while hopping
Data produced	12 pilot captures, 18.6 min, 3 people, 2 rooms	1 usable capture, 4,688 rows, 40 min, unusable for HAR
Used for the HAR results?	**Yes — all of them**	**No**

[M] The decisive check. Every file in `data/pilot/` — all twelve, including the four traces printed on deck pages 16 and 17 and the entire subcarrier analysis on page 14 — has the header `Packet,Timestamp_us,SC0_Re,…`. **No `Channel` column.** The Setup-2 extractor unconditionally writes a `Channel` column (`extract_csi_impl.cc:132`). Therefore **the pilot dataset was produced by Setup 1, the single-USRP passive sniffer** — not by the two-USRP system the review is about.

Reproduce it in front of your mentor if you want:

```
head -1 data/pilot/P01_action_amrith_20260717_walk.csv | cut -c1-45
head -1 data/3ch_test_20260813_001142.csv          | cut -c1-45
```

Why this matters more than anything else in this document: if you say "this is the data from our new two-USRP setup" while pointing at page 14 or 16, and your mentor opens the CSV, the review is over. Section 13 gives you the exact wording that is both honest and impressive.

1.3 What was genuinely achieved this phase — the honest claim

[V] All of the following are real, verified, and yours to claim:

1. A **dedicated 802.11a/g transmitter** built on a second USRP N210, emitting frames at a rate you choose (200 pps default), with a fixed synthetic MAC pair so the receiver can prove which frames are yours.
2. A **channel-aware CSI extractor** — the `extract_csi` GNU Radio block extended with a `Channel` column and a `set_channel()` entry point.
3. **Coordinated frequency hopping** across channels 1/6/11 on both radios, with an environment-variable park mode on each side.
4. **A real bug fix without which nothing worked at all**: the address filter looked only for the `ta` / `ra` metadata keys, which GNU Radio emits only for Control frames. Our transmitter sends Data frames. Every frame was being silently discarded.
5. **A measured, quantitative feasibility verdict on channel hopping** — and the finding is negative. This is a genuine engineering result, not a failure.
6. **A subcarrier-selection study** on the pilot data with leave-one-subject-out validation, showing that 8 of the 52 subcarriers beat all 52.

1.4 The 30-second version

> "We're building activity recognition for a smart home using WiFi channel state information instead of cameras, for users who sign or who can't hear. This phase we replaced a passive sniffer with a controlled two-USRP link — we now own the transmitter, so packet rate and channel are parameters instead of accidents. We also measured that three-channel hopping delivers only about 2 Hz of CSI against the 50-plus hertz activity recognition needs, so our recommendation is to record each channel separately rather than hop during an activity."

---

2. Problem Statement

2.1 As stated on the deck [V]

> Existing HAR systems rely on intrusive cameras, obtrusive wearables, or uncontrolled WiFi traffic. A controlled CSI framework is required to study privacy-preserving HAR and frequency-diverse WiFi sensing.

2.2 How to deliver it — read it as three rejections

This sentence is built as three separate failure modes. Delivering it that way is far stronger than reading it.

Approach	Fails on	Why it matters for *this* project
Cameras	Privacy	The rooms where falls happen — bathroom, bedroom — are exactly the rooms nobody accepts a camera in.
Wearables	Compliance	The device must be worn and charged. For the elderly-fall case (RO1) the dominant failure is the pendant left on the nightstand.
Uncontrolled WiFi traffic	Measurement quality	You do not own the sampling process. A physical measurement whose sampling rate is set by whichever app is running is not a measurement.

CSI over a controlled SDR link is the only option that clears all three at once. **That is the entire justification for the project in one sentence.**

2.3 The problem *underneath* the problem — say this if you want to sound senior

Sign-language recognition alone is useless in a home, because a recogniser needs to know **when to start listening**. A hand raised to sign HELLO and a hand raised to wipe a shelf produce similar gross motion. Before you can classify a sign you must **segment** it out of continuous ordinary living.

That is why the pilot dataset's classes are `action` vs `noaction` and not a set of signs. [V] — `USRP_lib/PILOT_ACTION_VS_NOACTION.md` states it explicitly: *"The person is present in both classes. That is deliberate: the model must learn motion, not presence."*

---

3. Project Objectives

3.1 Completed this phase (deck page 5) [V]

Objective	Status in the repository	Evidence
Controlled CSI acquisition — dedicated SDR TX–RX	**Done, hardware-verified**	`transmitter/…/wifi_tx_multi_hop.py` , `receiver/…/wifi_rx.py` ; matched frames observed
Channel diversity — cycle non-overlapping channels	**Done**	`3ch_test_20260813_001142.csv` contains all three channels: 1,555 / 1,734 / 1,399 rows [M]
CSI processing — extract, label, store with active channel	**Done**	`extract_csi_impl.cc` writes `Channel` per row
HAR — classify from temporal CSI variation	**Done offline, on Setup-1 data**	`analysis/sc_select.py` , `Action_NoAction_imgfi.ipynb`

Precision that protects you: acquisition, channel labelling and storage are verified against hardware. HAR is **offline, on captured CSVs** — there is no live classifier on the stream. Say that plainly; it is entirely normal at this stage.

3.2 Remaining (deck page 6) [V]

- **RO1** — WiFi-based sensing for reliable **fall detection**.
- **RO2** — multi-class activity data + **identify the most informative subcarriers**.
- **RO3** — WiFi-based **sign language identification** + classification model.
- **RO4** — **multimodal** ML integrating WiFi with complementary modalities.

Do not read this as a to-do list. RO2 is *partially delivered* — `analysis/sc_select.py` and `sc_scores.json` are exactly "identify the most informative subcarriers", validated leave-one-subject-out across 3 people and 2 rooms. Saying so converts a slide of promises into a slide with a completed line item.

---

4. Complete System Understanding — the repository, mapped

4.1 Top-level layout [V]

```
usrp-projects/
├── USRP_lib/                    SETUP 1 — single-USRP passive sniffer
│   ├── csi_tools/               the project's own Python pipeline
│   │   ├── session_logger.py    marks rep start/end -> *_labels.csv
│   │   ├── segment_windows.py   CSV -> 52 x N amplitude windows (.npy)
│   │   ├── segment_all_pilot.py batch driver for the pilot captures
│   │   ├── make_images.py       window -> 52 recurrence images (224x224)
│   │   ├── train_imgfi.py       ImgFi CNN, subject-split enforced
│   │   ├── train_hybrid.py      CNN-GRU-Attention model
│   │   └── check_capture.py     QA: rate, gaps, NaN/zero cols, wiggle plot
│   ├── gnuradio/gr-ieee802-11/  vendored OOT module; ONE block modified
│   ├── usrp/uhd/                vendored Ettus UHD driver (reference only)
│   ├── 1_DATA_CAPTURE_GUIDE.md  2_TRAINING_GUIDE.md  3_COMMANDS_CHEATSHEET.md
│   ├── CONTEXT.md               authoritative description of Setup 1
│   ├── PILOT_ACTION_VS_NOACTION.md   the pilot protocol
│   └── ImgFi_...pdf             the ImgFi paper itself
│
├── Three_Channel_Experiment/    SETUP 2 — two-USRP controlled link
│   ├── transmitter/             ships to the TX laptop
│   │   └── gr-ieee802-11/examples/
│   │       ├── wifi_tx_multi_hop.py   TX flowgraph + hop thread
│   │       └── packet_generator.py    traffic source, 200 pps
│   ├── receiver/
│   │   ├── gr-ieee802-11/lib/extract_csi_impl.cc   THE core custom block
│   │   ├── gr-ieee802-11/examples/wifi_rx.py       RX flowgraph + hop thread
│   │   ├── capture.sh                              labelled recording
│   │   └── RECEIVER.md
│   ├── doc/THREE_CHANNEL_ANALYSIS.md   feasibility study of hopping
│   ├── DATA_COLLECTION.md               protocol + the "don't hop" verdict
│   └── README.md
│
├── Three_Channel_Receiver/      teammates' raw drop; older ZMQ attempt (dead end)
├── gr-ieee802-11/, gr-foo/      TX-side upstream source at repo root
│
├── analysis/                    measurement + subcarrier selection
│   ├── analyze_pilot.py  -> pilot_stats.json
│   ├── make_figures.py   -> figures/fig1..10.pdf
│   ├── comparative_analysis.tex/.pdf   our pilot vs ARIL/UT-HAR/Widar3.0
│   ├── sc_features.py    -> sc_features.npz   (stage 1: features)
│   ├── sc_select.py      -> sc_scores.json    (stage 2: scoring + validation)
│   ├── sc_figures.py     -> figures/sc_fig1..10
│   ├── sc_report.py      -> sc_report.html
│   └── md2pdf.py         markdown -> PDF via headless Chrome
│
├── data/
│   ├── pilot/  12 x P01_*.csv   SETUP 1 data — all HAR results come from here
│   ├── csi_2026081*.csv         SETUP 2 parked attempts (0 rows)
│   └── 3ch_test_2026081*.csv    SETUP 2 hopping tests (1 usable)
│
├── Action_NoAction_imgfi.ipynb  ResNet18 + recurrence plots, LOSO
├── imgfi_model_fold1.pth.zip    44.8 MB trained weights
└── ppt_images/                  the 4 CSI graphs used on deck pages 16-17
```

4.2 Classification by viva relevance

A — Directly relevant (you must be able to explain these): `extract_csi_impl.cc`, `wifi_rx.py`, `wifi_tx_multi_hop.py`, `packet_generator.py`, `sc_features.py`, `sc_select.py`, `data/pilot/`, `DATA_COLLECTION.md`, `THREE_CHANNEL_ANALYSIS.md`.

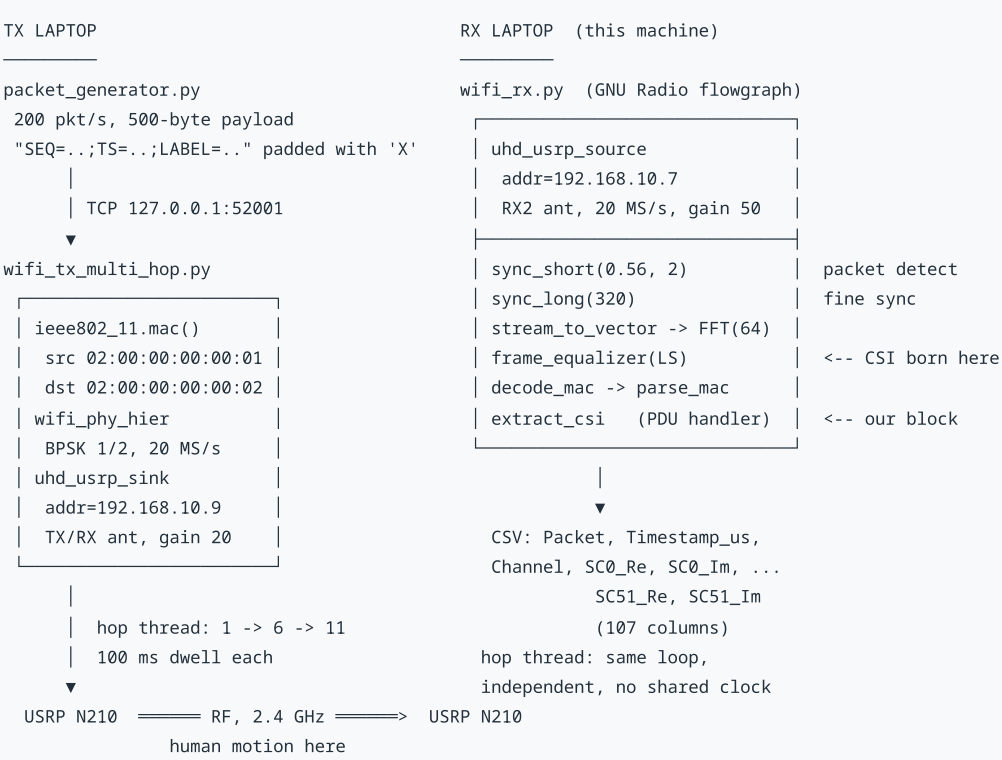
B — Supporting technical detail (needed to defend, not on slides): `lib/equalizer/ls.cc` and `base.cc` (where CSI actually comes from), `capture.sh`, `analyze_pilot.py`, `comparative_analysis.tex`, `segment_windows.py`.

C — Repository-only, plausibly asked (Section 17 covers these): `Action_NoAction_imgfi.ipynb` (ResNet18, not a lightweight CNN), `train_hybrid.py` / the CNN-GRU-Attention model, `check_capture.py`, the `--rate 100` window-drop bug, `sc_report.html`, the ZMQ dead end in `Three_Channel_Receiver/`.

D — Irrelevant (ignore): vendored `usrp/uhd/` clone, `gr-foo` internals, GNU Radio Qt GUI boilerplate, build artefacts, `texput.log`.

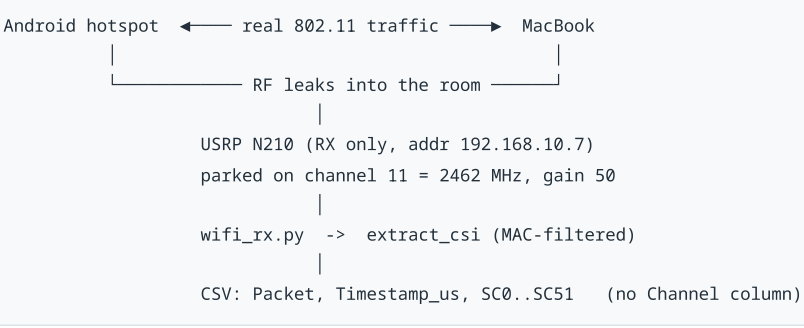
5. Architecture

5.1 Setup 2 — the architecture the deck presents [V]



Each USRP sits on its own laptop over a **direct point-to-point ethernet cable**. Both laptops use `192.168.10.1/24` on their own link — safe precisely because they are electrically separate links, not a shared LAN. **[V]** `Three_Channel_Experiment/README.md`

5.2 Setup 1 — the architecture that actually produced the results [V]



The USRP transmits nothing. It overhears an existing link and estimates the channel between the *hotspot* and *its own* antenna. That is a perfectly valid sensing channel — the limitation is that packet timing belongs to whatever app is running. **[V]** `USRP_lib/CONTEXT.md`

5.3 Component-by-component

`packet_generator.py` — the illumination source [V]

- **Does:** opens a TCP socket to `127.0.0.1:52001` and writes a 500-byte payload every `1/PPS` seconds. Payload is `SEQ=<n>;TS=<ms>;LABEL=<label>;` padded with `X`.
- **In:** environment `PPS` (default 200), `PAYLOAD_SIZE` (500), `LABEL`.
- **Out:** a byte stream into the GNU Radio TCP source inside the TX flowgraph.

- **Why it exists:** CSI only exists when a frame is transmitted. No packets, no measurement. This block *is* the sampling clock of the whole sensing system.
- **If it fails:** the flowgraph runs, the USRP transmits nothing, and the receiver's CSV stays empty. This is the first thing to check when a capture is empty.
- **Comment in the file, verbatim [V]:** *"PPS is the ceiling on the receiver's CSI sampling rate — the RX can never log more rows/s than the TX sends packets/s."*

`wifi_tx_multi_hop.py` — transmitter flowgraph [V]

- `ieee802_11.mac([0x02,0,0,0,0,0x01], [0x02,0,0,0,0,0x02], [0xff]*6)` — builds the 802.11 MAC header: source, destination, broadcast BSSID.
- `wifi_phy_hier(encoding=BPSK_1_2, bandwidth=20e6)` — scrambling, convolutional coding, interleaving, BPSK mapping, IFFT, cyclic prefix, preamble insertion.
- `uhd_usrp_sink(addr=192.168.10.9)`, antenna `TX/RX`, `tx_gain=20`, `samp_rate=20e6`.
- A daemon thread runs the hop loop or parks on `TX_CHANNEL`.

`wifi_rx.py` — receiver flowgraph [V]

Signal chain, in order: `uhd_usrp_source (RX2, 20 MS/s, 20 MHz BW, gain 50)` → normalised autocorrelation branch (`delay` + `moving_average` + `divide`) → `sync_short(0.56, 2)` → `delay` → `sync_long(320)` → `stream_to_vector(64)` → `fft(64)` → `frame_equalizer(Equalizer(chan_est), freq, samp_rate)` → `decode_mac` → `parse_mac` → **message port** → `extract_csi`.

Note the last hop is a **message (PDU) connection**, not a stream connection — `extract_csi`'s `work()` is a deliberate no-op stub and all logic lives in `handle_pdu`. [V]

`extract_csi_impl.cc` — the project's core custom block [V]

- **In:** a PDU whose metadata dictionary carries the `csi` key (52 complex numbers) plus address keys.
- **Processing:** looks up `ta`, falling back to `address 2`; looks up `ra`, falling back to `address 1`; lower-cases both; compares against `CSI_TA` / `CSI_RA` (or accepts anything if either is `any`); prints each distinct pair once with `(MATCH)` / `(no match)`.
- **Out:** one CSV row — `Packet, Timestamp_us, Channel, SC0_Re, SC0_Im … SC51_Re, SC51_Im`. Flushed every row.
- **Config:** entirely by environment variable — `CSI_TA`, `CSI_RA`, `CSI_OUT`. Never recompile to change a MAC.
- **Opens the CSV in append mode** and only writes a header if the file is new — so pointing `CSI_OUT` at an existing file never destroys it.

---

6. End-to-End Flow — follow one packet all the way through

This is the answer to *"one input enters the system — tell me exactly what happens to it."* Memorise this; it is the highest-value single answer in the viva.

Scenario: Amrith is walking across the room. The transmitter is parked on channel 11 (2462 MHz).

1. **A packet is born.** `packet_generator.py` builds `SEQ=4212;TS=1784270337545;LABEL=walk;` padded with `X` to 500 bytes, and writes it to TCP `127.0.0.1:52001`. [V]
2. **MAC framing.** `ieee802_11.mac` prepends an 802.11 header: frame control, duration, address 1 = `02:00:00:00:00:02`, address 2 = `02:00:00:00:00:01`, address 3 = broadcast BSSID, sequence control. It appends a 32-bit FCS. [V]
3. **PHY modulation.** `wifi_phy_hier` scrambles the bits, convolutionally encodes at rate 1/2, interleaves, maps to BPSK, loads 48 data subcarriers plus 4 pilots into a 64-point IFFT, prepends a 16-sample cyclic prefix per symbol, and prepends the preamble — 10 short training symbols then 2 long training symbols. [V] + [S]
4. **Over the air.** The USRP upconverts to 2462 MHz and transmits at gain 20. The signal reaches the receive antenna by many paths: line-of-sight, floor bounce, wall bounces, and **a reflection off Amrith's moving torso and limbs**. Each path has its own delay, so each arrives with its own phase; they sum vectorially at the antenna. [S]
5. **Reception.** The RX USRP downconverts a 20 MHz slice centred on 2462 MHz and streams complex baseband to the host at 20 MS/s over gigabit ethernet. [V]
6. **Packet detection — `sync_short`.** The 10 short training symbols repeat every 16 samples, so a normalised autocorrelation at lag 16 spikes at the start of a frame. Threshold `0.56`, minimum plateau `2`. This also drives AGC and a coarse frequency-offset estimate. [V] threshold + [S] mechanism
7. **Fine sync — `sync_long`.** Cross-correlates against the known long training sequence over a `320`-sample window to find the exact symbol boundary and estimate fine carrier frequency offset. [V]
8. **FFT.** Each 64-sample OFDM symbol goes through a 64-point FFT, producing 64 frequency bins. [V]
9. **Channel estimation — where CSI is born.** In `lib/equalizer/ls.cc`, the least-squares equaliser stores the first long training symbol, then on the second computes, for each used bin `i`: [V] — this is literally the code

$$H[i] = (LTF_1[i] + LTF_2[i]) / (2 \cdot L[i])$$

where `L[i]` is the known transmitted LTF value (±1). Averaging the two repetitions gives a 3 dB noise reduction; their *difference* gives the SNR estimate `d_snr`. Data symbols are then equalised as `symbols[c] = in[i] / d_H[i]`.

10. **Subcarrier selection.** `base::get_csi()` copies out bins `6` to `58` **excluding** bin `32` (DC) — exactly **52 values**: 48 data + 4 pilots. Guard bands are discarded. **[V]**
11. **MAC decode and address filter.** `decode_mac` checks the FCS and recovers the header; `parse_mac` publishes a PDU with metadata. `extract_csi::handle_pdu` reads `csi`, then `address 2` / `address 1`, compares against `CSI_TA` / `CSI_RA`. Amrith's frame matches. **[V]**
12. **Persisted.** One CSV row is written and flushed: packet counter, host wall-clock microseconds, the currently tuned channel, and 104 floats (52 real + 52 imaginary). **[V]**
13. **Offline — features.** `sc_features.py` reads the CSV, computes `amp = hypot(Re, Im)` and converts to dB, de-spikes samples more than 6 MAD from a 5-tap running median, linearly interpolates onto a uniform **80 Hz** grid, cuts 2-second windows with a 1-second hop, and per (window, subcarrier) computes 7 dynamics features. **[V]**
14. **Offline — selection.** `sc_select.py` scores each subcarrier by one-vs-rest separability and by motion-SNR profile, takes the **worst case across the two rooms**, de-duplicates with mRMR, then validates with a random forest under **leave-one-subject-out**. **[V]**
15. **Output.** A predicted activity label for that 2-second window — plus, in the intended deployment, a downstream decision such as "signing has started, hand the stream to the sign recogniser". **[V] for the classifier, [L] for the deployment step — it is not implemented.**

---

7. Slide-by-Slide Explanation

The PDF has **20 pages**. Frame numbering on the slides runs 1, 2, 3, 4, 4, 5 … 12, 12, 13 … 16 — **two frames are numbered 4 and two are numbered 12.** See Section 13 for the fix list.

Slide 1 (p1) — Title

What is shown. "Type 2: Application Project". Title, course code 23CSE399 Project Phase 2, "Guide Review 1 | Panel Review 1", five team members, guide, department.

What should I say.

> "This is an application project — we're building a camera-free sensing layer for a smart home serving two groups: users who sign, and users who can't hear events happening around them. Both need the same thing underneath, which is continuous activity sensing that doesn't need a camera or a worn device."

What it technically means. "Type 2 / Application" commits you to being judged on whether the system *works and is honestly measured*, not on methodological novelty. That is a favourable frame — lean into it.

Why included / why chosen. The two user groups are genuinely different problems sharing one substrate: a speech-impaired user *produces* sign language (gesture recognition); a hearing-impaired user *receives* information (event detection surfaced visually).

Likely questions.

- *"Why are these two groups in one project?"* → Both need continuous, non-visual, non-worn sensing of a room. The sensing layer is shared; only the downstream classifier differs.
- *"Which of the two are you actually building first?"* → The segmentation layer that both need — telling motion from stillness, and eventually signing from ordinary activity. **[V]** That is what the `action` / `noaction` pilot is.

Slide 2 (p2) — Use Cases: UC1 and UC2

What is shown. UC1 Daily Activity Recognition (walking, sitting, cleaning; cameras intrusive; standard sensors lack fine-grained motion; CSI captures propagation changes). UC2 Activity → Sign Transition (user moves from household activity to signing; gestures confused with ordinary movement; CSI gives fine-grained temporal data).

What should I say.

> "UC1 is the general capability. UC2 is the one that actually drives the design. A sign recogniser is useless in a home unless something tells it *when* signing has started — raising a hand to sign and raising a hand to clean a shelf look similar in gross motion. So the first problem isn't recognition, it's segmentation, and that's exactly what our pilot dataset tests: action versus no-action, with the person present in both classes."

What it technically means — the physics, in one paragraph. [S] A WiFi signal does not travel along one path. It arrives over the direct line plus reflections off walls, floor, ceiling, furniture and the human body. Each path has its own length, hence its own delay and phase, and they sum at the receive antenna. When a person moves, reflected path lengths change by centimetres — and at 2.462 GHz the wavelength is **12.2 cm**, so a few centimetres is a large fraction of a cycle. The sum therefore changes continuously and measurably. CSI is that sum, measured per subcarrier per packet.

Why the privacy claim holds — and its honest limit. What is *not* in the recording: no image, no audio, no identity. 52 complex numbers describing a radio channel. [V] But be careful: [S] the published literature shows CSI can support keystroke and gait inference under favourable conditions. The defensible claim is *"substantially less intrusive than a camera"*, not *"zero information"*.

Alternatives considered. [L] PIR sensors — cheap but binary presence only, no motion detail. Radar/mmWave — better resolution, but dedicated hardware and cost. Ultrasound — short range, occlusion-sensitive. WiFi CSI reuses infrastructure that is already in every home. The repository does not contain a written trade study, so present this as *our* reasoning, not as a documented comparison.

Likely questions.

- *"How is this different from a motion sensor?"* → A PIR gives one bit. CSI gives a 52-dimensional complex vector at 50–80 Hz, which resolves *what kind* of motion, not just that there was some.
- *"Can it work through walls?"* → [S] 2.4 GHz penetrates plasterboard with attenuation; the signal degrades but does not vanish. **We have not tested through-wall — do not claim it.**
- *"How many people can it handle?"* → One. [V] Every capture is single-occupant. Multi-person separation is an open problem and is why "wearable identity tagging" is on the future slide.

Slide 3 (p3) — Use Case UC3: Multi-Channel CSI

What is shown. UC3: monitoring across multiple non-overlapping channels; sequential hopping 1/6/11 at 100 ms dwell; purpose is frequency diversity; limitation stated as sequential rather than simultaneous. Key insight: controlled SDR TX–RX lets channel, packet generation and sensing procedure be custom-designed for HAR.

What should I say.

> "Channels 1, 6 and 11 are the only three in the 2.4 GHz band that don't overlap, so they give three genuinely independent views of the same motion. The idea is frequency diversity — if the body's reflection happens to land in a fading null at one frequency, another frequency will still see it. We implemented the hopping on both radios. I should say up front that we then measured what it costs, and the answer changed our plan — I'll cover that on the limitations slide."

Why 1, 6 and 11 specifically. [S] 2.4 GHz 802.11 channel centres are 5 MHz apart, but each channel is 20 MHz wide. Channel 2 overlaps channel 1, channel 3 overlaps both, and so on. Channels 1 (2412 MHz), 6 (2437 MHz) and 11 (2462 MHz) are the only trio whose 20 MHz occupancies do not overlap at all — centres 25 MHz apart, so 20 MHz occupancies leave 5 MHz clear between them.

What frequency diversity buys. [S] Multipath fading is *frequency-selective*: two paths arriving half a wavelength apart cancel, and whether they cancel depends on frequency. The channel therefore has deep nulls at some frequencies and peaks at others. Channels 1 and 11 are 50 MHz apart — far more than the coherence bandwidth of a typical indoor room — so their fading patterns are effectively independent.

Why 100 ms dwell? [V] + [L] `DWELL_TIME = 0.1` is hard-coded identically in both `wifi_tx_multi_hop.py` and `wifi_rx.py`. The repository does **not** document why 100 ms was chosen initially — treat it as an initial guess. What *is* documented is the measurement that followed: roughly **60 ms of every 100 ms dwell is lost to settling** (`DATA_COLLECTION.md`), or ~110 ms in the independent estimate in `THREE_CHANNEL_ANALYSIS.md`. Either way the dwell is comparable to or shorter than the settling time.

Likely questions.

- *"What is the actual CSI rate while hopping?"* → **1.96 Hz [M]**, measured over 2,394 s of `3ch_test_20260813_001142.csv`. Against 51–81 Hz parked.
- *"Would 50 ms dwell be better?"* → **Strictly worse. [V]** The retune cost is fixed *per hop*, so halving the dwell doubles the number of hops and doubles total overhead. At 50 ms the settling exceeds the entire dwell and nothing gets through.
- *"Why not hop faster, like published systems?"* → [V] `THREE_CHANNEL_ANALYSIS.md` cites published systems hopping within the channel coherence time with hop delays as low as 0.2 ms — roughly **500× faster than an N210 can retune**. The technique is sound; this hardware cannot execute it.

Slide 4 (p4) — Motivation: five reasons

What is shown. (1) Privacy — no cameras. (2) Fine-grained motion — amplitude and phase shifts. (3) Controlled acquisition — no dependence on unpredictable phone/laptop traffic. (4) Frequency diversity. (5) Reproducibility — control over TX parameters, channel sequence, dwell. Key insight: SDR gives direct programmatic control over physical-layer sensing.

What should I say — spend your time on point 3.

> "Points 1, 2 and 4 are arguments for WiFi sensing in general. Point 3 is the argument for *this semester's rebuild*, and it's the strongest one. Previously our sampling rate was whatever the laptop's traffic happened to be — a video call gives a torrent, an idle screen gives nothing. That means the sampling rate of a physical measurement was being set by an app. Human limb motion has energy up into the tens of hertz, so we need a guaranteed rate and roughly uniform spacing. Owning the transmitter is the only way to get that."

Technical depth on point 3. [S] + [V] Two separate requirements are hiding in "controlled acquisition":

1. **Rate (Nyquist).** At 2.462 GHz the Doppler shift is $2 f_c / c = 16.4$ Hz per m/s of radial speed **[M, computed]**, so a torso at ~1 m/s lands near 16 Hz and limb tips higher. You need a comfortable margin above that.
2. **Uniformity.** Every standard CSI tool — STFT, wavelet, sliding-window PCA — assumes uniform sampling. **[V]** `sc_features.py` has to interpolate onto an 80 Hz grid precisely because arrival is bursty.

Point 5, reproducibility, restated usefully. [L] With a controlled transmitter the experiment becomes a *defined object*: same room, same pps, same payload, same channel sequence, same dwell. Another group can re-run it. Under passive sniffing there is no experiment to re-run, only a recording of an afternoon.

Likely questions.

- *"Point 2 says phase. Do you use phase?"* → **No. [V]** Amplitude only — see Slide 12 below for the full answer. This is the most likely trap on this slide.
- *"How much control do you actually have?"* → **[V]** `PPS`, `PAYLOAD_SIZE`, `LABEL`, `TX_CHANNEL`, `CSI_CHANNEL`, `tx_gain`, `encoding`, plus the MAC pair. All settable without recompiling.

Slide 5 (p5) — Problem Statement & Completed Objectives

Covered fully in Sections 2 and 3.1 above. Deliver the problem statement as **three rejections** (cameras/privacy, wearables/compliance, uncontrolled traffic/measurement quality).

Likely questions.

- *"What exactly is completed?"* → Acquisition, channel labelling, CSV storage — hardware-verified. HAR — offline on captured CSVs, no live classifier.
- *"Show me the controlled acquisition."* → `Three_Channel_Experiment/transmitter/gr-ieee802-11/examples/wifi_tx_multi_hop.py` plus `packet_generator.py`; the receiving half is `receiver/gr-ieee802-11/examples/wifi_rx.py`.

Slide 6 (p6) — Objectives to be Completed: RO1–RO4

Covered in Section 3.2. **Key move: say RO2 is partially delivered and point at Slide 12.**

Why each objective exists. [L] except where noted

- **RO1 fall detection** — the highest-value clinical case, and the one where a wearable most reliably fails. A fall is also a favourable signal: short, high-energy, abrupt.
- **RO2 subcarrier selection** — feature reduction. 52 subcarriers × 7 features = 364 dimensions on 1,093 windows **[V]** is an overfitting recipe. Selection is what makes the problem tractable.
- **RO3 sign identification** — the hard one. Signs are small, fast, hand-scale motion; walking is large, slow, whole-body. Expect to need higher packet rates and probably phase.
- **RO4 multimodal** — the hedge, and what makes "identity-aware" possible.

Likely questions.

- *"Which of these is realistic by end of Phase 2?"* → RO2 is largely done. RO1 and the first stage of RO3 are achievable after the parked-per-channel recapture. RO4 needs the wearable stream time-aligned, which has not been started. **[V]**
- *"What's the novelty?"* → Honest answer: the novelty is not the algorithm. It is (a) a fully controlled SDR sensing link where every acquisition parameter is ours, and (b) a measured, negative result on channel hopping with an N210, which the literature does not report because published systems assume hardware that retunes 500× faster.

Slide 7 (p7) — Literature Survey

What is shown. Four papers in a table.

Paper	Method	Result	Limitation
SignFi (2018)	9-layer CNN on CSI amplitude and phase	276 signs, 98% lab	falls to **67%** for unseen users

Paper	Method	Result	Limitation
Widar3.0 (2021)	Body-coordinate Velocity Profile + CNN–GRU	83–92% cross-domain	needs LOS and multiple synchronized links
ImgFi (2023)	CSI → recurrence-plot images → lightweight 2D-CNN	99.5–99.8%, ~90× fewer params than ResNet50	within-domain only
Joint Activity & Localization (2019)	Dual-branch 1D-CNN (ResNet1D) on subcarrier fingerprints	88% activity, 95.7% location	single user, single room

What should I say — the takeaway line.

> "The number to notice is SignFi's: 98% in the lab, 67% on a user the model hadn't seen. Most headline accuracies in this field are within-domain. That's why we chose leave-one-subject-out from the start — our numbers are lower and they mean more."

Explain BVP if asked about Widar3.0. [S] Widar3.0's contribution is that it refuses to classify CSI directly. Raw CSI is a property of the *room* — train on room A, learn room A. Widar combines Doppler measured on several links into a velocity profile expressed in the **body's own coordinate frame**, which is a physical quantity about the person, not the room, so it transfers. The price is multiple synchronized links. **You have two USRPs, one of which transmits — so this approach is closed to you on current hardware.** That is the honest reason you did not adopt it.

Explain recurrence plots if asked about ImgFi. [S] + [V] A recurrence plot turns a 1-D time series into a 2-D image where pixel (i,j) is bright when the signal state at time i is close to its state at time j. Periodic motion — a gait cycle — appears as regular diagonal stripes. Once it is an image you can use a cheap 2-D CNN. **[V]** Your implementation uses the distance form $RT[i,j] = |x_i - x_j|$ (`recurrence_plot()` in `Action_NoAction_imgfi.ipynb`, and `make_images.py`).

CRITICAL — the trap on this slide. [V] The slide credits ImgFi with *"~90× fewer parameters than ResNet50"*. **Your own implementation does not do that.** `Action_NoAction_imgfi.ipynb` uses `models.resnet18(weights=ResNet18_Weights.DEFAULT)` — a pretrained ResNet18, ~11.7 M parameters, on 224×224 inputs. The saved checkpoint is 44.8 MB, consistent with that. `USRP_lib/csi_tools/train_imgfi.py` is the 3-conv-layer lightweight version described in `CONTEXT.md`, but the notebook at the repository root — the one whose results you would quote — is transfer learning on ResNet18.

How to handle it if asked "is your model lightweight?":

> "Two implementations exist. `csi_tools/train_imgfi.py` follows the paper with a small three-convolution CNN. The notebook we ran the pilot results on uses a pretrained ResNet18 instead, because with only about 300 windows, ImageNet initialisation converged far more reliably than training a small net from scratch. So the accuracy we quote is not a lightweight-model result — it's transfer learning, and the lightweight comparison is still to be done."

Likely questions.

- *"Why these four papers?"* → They bracket the space: SignFi = the target task with the domain-shift problem; Widar3.0 = the cross-domain solution and its hardware cost; ImgFi = the representation we adopted; Joint Act.&Loc. = multi-task on subcarrier fingerprints.
- *"Where is ImgFi in your reference list?"* → **It is missing.** Refs 1–11 do not include it. Fix before submission. **[V]**

Slide 8 (p8) — System Evolution: Passive → Controlled

What is shown. Setup 1: phone hotspot → laptop WiFi traffic → single USRP passive capture (passive observation, traffic dependent, limited channel control). Setup 2: USRP N210 TX → wireless channel → USRP N210 RX → CSI extraction (controlled transmission, dedicated TX/RX, channel hopping).

This is the slide the review is about. Give it more time than any slide except 12.

What should I say.

> "Last phase we were a third radio listening to somebody else's link. Physically that's fine — the CSI we measure is the channel between the hotspot and our own antenna, which is a valid sensing channel. The problem is everything we had no say over: when a packet was sent, at what rate, on which channel, with what payload, at what modulation. This phase we own both ends, which turns every one of those from an uncontrolled variable into a parameter we set."

The concrete comparison — memorise this table. [V]

Property	Setup 1 (passive)	Setup 2 (controlled)
Packet rate	whatever the app does	`PPS=200`, `packet_generator.py`
Channel	wherever the hotspot sits (was ch 11)	1 / 6 / 11, chosen via `TX_CHANNEL` / `CSI_CHANNEL`
Source identity	real device MAC, can collide with other traffic	`02:00:00:00:00:01` → `02:00:00:00:00:02`

Property	Setup 1 (passive)	Setup 2 (controlled)
Payload	variable	`PAYLOAD_SIZE=500`, fixed
Modulation	whatever rate control picks	`BPSK_1_2`, fixed
Delivered CSI rate	51–81 Hz (opportunistic)	1.96 Hz hopping; parked mode not yet run
Reproducible	no	yes — every parameter declared

Why a synthetic MAC pair? [V] + [L] `02:` as the first octet sets the *locally administered* bit, so the address is guaranteed not to collide with any real manufacturer-assigned MAC. **[S]** With a fixed known pair the receiver can filter exactly and guarantee every CSV row came from your experiment; under passive capture other traffic on the same channel can slip past a loose filter and pollute the dataset.

Likely questions.

- *"Why is BPSK 1/2 used — that's the slowest rate?"* → **[V]** `encoding = ieee802_11.BPSK_1_2`. **[L]** BPSK rate 1/2 is the most robust 802.11a/g modulation, so frames decode at the lowest SNR. For sensing you do not care about throughput — you care that *every* frame decodes, because a frame that fails its FCS produces no CSI row. Choosing the most robust rate maximises the CSI sampling rate.
- *"Does the receiver transmit anything?"* → No. It never associates, never ACKs. "Active sensing" here means *we control the illumination*, not that the receiver participates in a network.

Slide 9 (p9) — Two-USRP CSI Setup

What is shown. TX (IEEE 802.11) → wireless channel (human motion) → RX (IQ capture). Channel 1 = 2412 MHz, 6 = 2437, 11 = 2462. Controlled transmission, dedicated TX/RX, sequential hopping, ≈100 ms dwell.

The physical build. [V] Two laptops, one USRP each, direct point-to-point ethernet per pair. Both laptops use `192.168.10.1/24` locally — safe because the links are electrically separate. TX USRP at `192.168.10.9`, RX at `192.168.10.7`. `packet_generator.py` feeds `wifi_tx_multi_hop.py` over TCP `52001`.

THE question that is coming: "how are the two radios synchronized?"

Answer, delivered before they ask:

> "To be clear about a limitation on this slide — the two USRPs have no common clock. No GPSDO, no distributed 10 MHz reference. Each runs its own hop loop from whenever its process started, so they drift in and out of alignment and are on the same channel roughly a third of the time. And parking the receiver wouldn't help: a parked receiver also only sees a hopping transmitter a third of the time. Fixing it properly means a GPSDO or an external reference on each unit, and it's on our limitations slide."

[V] `Three_Channel_Experiment/README.md` states this explicitly, and `wifi_rx.py`'s header comment corrects an earlier mistaken figure: two free-running loops share a channel **~1/3** of the time, **not 1/9**.

Second-order consequence worth knowing. [S] No common local oscillator also means residual carrier frequency offset on every packet — which is *precisely why* the analysis is amplitude-only. Connecting those two facts in one sentence shows the limitation is understood, not just listed.

Likely questions.

- *"Why gain 50 on RX and 20 on TX?"* → **[V]** those are the values in the flowgraphs. **[L]** TX gain is kept modest because the radios are 3–5 m apart in the same room; more power would risk saturating the receiver front end and would also raise interference with real WiFi in the building. RX gain 50 is near the top of the N210's range to maximise SNR on a weak indoor link. **The repository does not document a gain sweep — do not claim one was done.**
- *"Why antenna RX2 on the receiver?"* → **[V]** `set_antenna("RX2", 0)`. **[S]** On Ettus daughterboards, `TX/RX` is a shared transmit/receive port and `RX2` is receive-only; using `RX2` avoids the T/R switch entirely on a receive-only node.

Slide 10 (p10) — CSI Acquisition Pipeline

What is shown. IQ samples → 802.11 sync (short + long) → frame equalization → CSI extraction; output described as complex channel response across OFDM subcarriers.

This is where a communications-literate panellist will probe. Know the six real stages — they are listed step by step in Section 6, items 5–12. Reproduce that chain.

The one-line explanation that lands best:

> "The channel estimate isn't something we compute separately — it's already there, because every 802.11 frame carries a known training field the receiver *must* use to equalise. We take the least-squares estimate the equaliser has already formed from the two long training symbols, and that's our CSI. We're reading out a by-product of normal WiFi demodulation."

The two fixes worth mentioning if asked "what took a semester?" [V]

1. **The silent drop.** `parse_mac` emits `ta` / `ra` metadata keys only for Control frames. Our transmitter sends Data frames, whose addresses appear as `address 2` / `address 1`. Before the fix the filter found neither key and **every frame was silently discarded** — the pipeline ran perfectly and produced an empty CSV.
2. **The buffering.** The CSV flushed every 100 rows. At hopping match rates that left the file looking empty for minutes, indistinguishable from a broken capture during a live session. It now flushes per row.

Likely questions.

- *"Why least squares and not something better?"* → **[V]** `chan_est = 0` selects LS; the flowgraph exposes four options — **LS, LMS, Linear Comb, STA**. **[S]** LS is unbiased and makes no assumption about the channel's structure; LMS and STA smooth across subcarriers or symbols, which *reduces noise but also suppresses exactly the fine per-subcarrier variation we are trying to sense*. **[L]** For sensing, an unsmoothed estimate is the right default. This is a strong answer — the alternatives are literally radio buttons in your GUI.
- *"Why 52 subcarriers, not 64?"* → **[S]** 802.11a/g uses a 64-point FFT but only 52 bins carry energy: 48 data + 4 pilots. The remaining 12 are guard bands and DC, which carry no transmitted signal, so no channel can be estimated there. **[V]** `get_csi()` keeps bins 6–58 excluding 32.
- *"Do you use the pilot subcarriers?"* → Yes — all 52 are kept, including the 4 pilots at bins 11, 25, 39, 53. For sensing there is no reason to discard them; they carry a channel estimate like any other.

Slide 11 (p11) — Current Capability vs Future Extension

What is shown. Current: two-USRP TX–RX, controlled transmission, CSI extraction, channels 1/6/11, sequential hopping, channel-labelled CSI. Future: sign-language gesture detection, wearable identity tagging, CSI + wearable fusion, multi-modal HAR, end-to-end recognition. Important note: hops between channels, does not receive 1/6/11 simultaneously.

Why that "Important" line exists — and the full answer behind it. [V] `THREE_CHANNEL_ANALYSIS.md` §6.3 answers this properly and it is one of the best answers in your whole repository:

- Channels 1 and 11 span **70 MHz** including channel widths (2402 → 2472 MHz).
- **Gigabit ethernet to the host** limits the N210 to ~25 MS/s → **~20 MHz usable**. Short by 3.5×.
- The **SBX daughterboard's analog filter is fixed at 40 MHz** (probed with `uhd_usrp_probe`; freq range 400–4400 MHz). Short by 1.75×.
- Your flowgraph already runs `samp_rate = 20e6` — **already at the platform's practical ceiling. There is no unused headroom.**
- **The constraint most people miss: the transmitter has the same limit.** CSI measures how a *known transmitted signal* was altered. Our transmitting N210 can illuminate only one 20 MHz channel at a time, so even a wideband receiver would see our packets on one channel and noise on the other two. **Wideband reception only helps if paired with wideband transmission.**
- **What would suffice:** a USRP X310 with a UBX-160 daughterboard — 160 MHz over 10 Gigabit Ethernet. That is a different hardware class, not a configuration change.

Say it as: *"It's a hardware-count and bandwidth limitation, not a software one — and our own flowgraph is already at the ceiling."*

"Wearable identity tagging" — what it means. [L] CSI tells you a body moved. It does not reliably tell you *which* body, and in a two-occupant home that ambiguity breaks the sign-language use case. Distinguishing occupants from CSI alone is an open research problem; a watch or phone IMU on a known person resolves it immediately. That is why it is under Future rather than something attempted and failed.

Slide 12 (p12) — CSI Data: From Channel Response to HAR Input

What is shown. Received frame → CSI → channel label (1/6/11). CSI representation: $H_k = |H_k|\, e^{j \angle H_k}$, "amplitude and phase provide the sensing information". HAR input: CSI arranged over time; input = temporal CSI variations, output = activity/sign class.

Unpack the equation properly. [S] + [V] H_k is one complex number for one subcarrier from one packet. 52 subcarriers gives a 52-element complex vector per packet. Stack N packets along time and you have a **52 × N matrix** — that matrix is the "image" every method in the literature ultimately consumes. **[V]** In `segment_windows.py` the window is 52 × 200 (2 s at 100 Hz); in `sc_features.py` it is 2 s at 80 Hz = 52 × 160; in the ImgFi notebook `RATE=50`, `WINDOW_SECONDS=2` gives 52 × 100.

AMPLITUDE VS PHASE — the highest-probability probe on this slide.

The slide says amplitude *and* phase. **Your pipeline uses amplitude only. [V]** — `amp = np.hypot(re, im)` in both `sc_features.py` and `segment_windows.py`; `CONTEXT.md` states outright: *"amplitude only — raw phase is unusable, the USRP isn't synced to the transmitter."*

Answer:

> "Phase carries more information in principle, and that's why the slide mentions it. But with two independent radios there's no common clock, so every packet's measured phase includes an unknown carrier frequency offset, a sampling frequency offset, and a packet-detection delay. Those are larger than the Doppler we're trying to measure, so raw phase is close to a random walk from packet to packet. Cleaning it up needs either a shared reference, or a second receive antenna so you can conjugate-multiply the offsets away. We have neither yet — so we work in amplitude, which is also what ARIL does with the same class of front-end."

Why the Channel column exists. [V] + [S] Because the receiver hops, a row's CSI is only comparable to rows on the *same* channel — 2412 MHz and 2462 MHz have different multipath responses and are, for sensing purposes, different signals. The `Channel` column is what lets the file be split back into three coherent streams. Without it the CSV would be three interleaved signals pretending to be one time series.

Likely questions.

- *"What is the wavelength?"* → **[M]** $\lambda = c/f = 3\times10^8 / 2.462\times10^9 = 12.2$ cm. Half-wavelength ≈ 6.1 cm is the displacement that flips a reflected path by 180°.
- *"What's the subcarrier spacing?"* → **[S]** 20 MHz / 64 = **312.5 kHz**.
- *"Which frequency is SC0?"* → **[V, from `base.cc`]** `get_csi()` keeps FFT bins 6–58 excluding 32, so SC0–SC25 map to offsets −26…−1 and SC26–SC51 to +1…+26 × 312.5 kHz. SC0 is 26 × 312.5 kHz = **8.125 MHz below** the channel centre.

Slide 13 (p13) — CSI Processing and Software Architecture

What is shown. Python Controller (hopping) → 802.11 PHY (GNU Radio) → USRP TX (UHD) → wireless → USRP RX (UHD) → 802.11 RX (sync) → CSI Extractor. TX side and RX side checklists.

Name the layers — it makes the slide land. [V]

Layer	What it is	Ours?
UHD	USRP Hardware Driver — Ettus's vendor driver; talks to the N210 over ethernet, does tuning and sample transport	No — vendored
GNU Radio	the dataflow framework the DSP graph runs inside (3.10.11.0)	No
gr-ieee802-11	Bastian Bloessl's OOT module: a full 802.11a/g PHY as GNU Radio blocks	No — extended
gr-foo	supporting blocks used by the above	No
`extract_csi` **block**	consumes the decoded-frame PDU, filters by MAC, writes CSV	**Yes**
`Channel` **column +** `set_channel()`	channel labelling	**Yes**
Hop loops, park modes, `capture.sh`	acquisition control	**Yes**
`csi_tools/` , `analysis/`	segmentation, features, selection, training	**Yes**

Say this out loud: *"We did not write an 802.11 PHY — we extended a well-known open-source one."* A panel will respect that far more than a vague implication that you built the radio stack.

How the hop is actually done. [V] In Python, not in the flowgraph: a daemon thread calls `set_center_freq` on the USRP source/sink and `set_channel()` on the extractor — so the CSV records which channel each row belongs to — then sleeps `DWELL_TIME` . `CSI_CHANNEL=1|6|11` parks the receiver; `TX_CHANNEL=1|6|11` parks the transmitter; unset means hop.

Likely questions.

- *"Why Python for the hop and C++ for the extractor?"* → **[L]** The hop is a slow control-plane operation (10 per second) where Python's overhead is irrelevant and iteration speed matters. The extractor sits in the per-packet data path inside GNU Radio's C++ block API, where it must not add latency — and it has to live in the flowgraph to receive PDUs at all.
- *"Why environment variables instead of command-line arguments?"* → **[V]** `CONTEXT.md` : *"never recompile to change a MAC or output path."* **[L]** The block is instantiated inside a GRC-generated flowgraph that would be overwritten on regeneration, so it has no clean argument surface; environment variables survive regeneration.

Slide 14 (p14) — Optimal Subcarrier Selection per Activity

This is your strongest technical result and the slide with the largest gap between what it shows and what it claims. Read Section 10.3 as well before the review.

What is shown. Four bar charts (Walking, Mixed, Standing, Sitting), each plotting **robust separability** (y, 0–1) against **subcarrier index** (x, 0–51), top ten highlighted. Figure title: *"worst-case of the two rooms"*. Top-10 lists per activity. Caption: *"ranked by how much they respond to motion versus noise, kept only if that held true across both rooms and all subjects, thinned to remove duplicates, and confirmed by testing on a person the model hadn't seen."*

Top 10 per activity [V], `sc_scores.json` **:**

Activity	Top 10 subcarriers
Walking	26, 38, 28, 39, 37, 12, 27, 36, 11, 13
Mixed	38, 37, 39, 47, 33, 8, 34, 46, 45, 48
Standing	6, 5, 19, 21, 20, 18, 22, 4, 23, 8
Sitting	2, 51, 42, 43, 5, 41, 40, 3, 44, 4

The dataset behind it. [V] + [M] 12 captures, 3 people, 2 rooms, 18.6 min, 1,093 two-second windows. Amrith and Rishiikesh in Room A on 17 July; Gowreesh in Room B on 21 July — four days apart, so his captures are simultaneously the cross-person *and* cross-room test.

How the numbers were produced. Full method in Section 10.3.

THE MISMATCH YOU MUST DECIDE HOW TO HANDLE

The **caption** describes the pipeline that produced the **recommended set**. The **figure** shows the **per-activity rankings**. Only the "both rooms" part of the caption applies to what is plotted — the thinning (mRMR) and the unseen-person validation produced a *different* answer, **SC20, 21, 22, 23, 27, 28, 29, 30**, which does not appear on the slide at all. **[V]**

This matters because your own reliability analysis says the per-activity rankings **do not transfer**: **[V]** `sc_scores.json → reliability`

Test	Walking	Mixed	Standing	Sitting
Split-half within one capture	+0.78	+0.72	+0.91	+0.87
Cross-subject, **same room**	**−0.24**	**−0.29**	**−0.43**	**−0.14**
Cross-room	+0.14	**−0.56**	+0.15	+0.10

The motion-SNR profile behaves the opposite way — for walking it holds at **ρ = +0.81 across people and +0.91 across rooms**. The recommended eight are built only from that second, transferable measurement.

So do not say "these are the subcarriers we will use." Your own data says the per-activity lists are a fact about one afternoon's multipath.

Recommended script (adds ~30 seconds, converts a gotcha into your best moment)

> "Each activity really does favour a different part of the band — walking peaks around subcarriers 26–28 and 36–39, standing sits low at 4–8 and 18–23, sitting favours the extreme edges. But we then tested whether those rankings survive a change of person, and they don't — they're actually *negatively* correlated between two people in the same room. What *does* survive is how strongly a subcarrier responds to motion relative to its own noise floor: that holds at rho above 0.8 across people and 0.9 across rooms. Selecting on that gives eight subcarriers — 20 to 23 and 27 to 30. On a person the model has never seen, those eight give 58% four-class accuracy against 42% using all 52. Cutting the input by 85% doesn't cost accuracy, it buys it."

The validation table — memorise the 8 and 52 rows [V]

Subcarriers	Selected, new person	Random set	Worst set	Same capture
1	41.5%	43.3%	35.3%	63.8%
4	54.8%	44.2%	42.9%	81.5%
8	**57.8%**	41.7%	35.2%	83.4%
12	55.5%	42.2%	35.4%	86.3%
20	54.1%	41.3%	38.4%	89.6%
52	**41.6%**	41.6%	41.6%	**93.6%**

The most instructive number in the project is the gap between the last two columns. 93.6% within a capture vs 41.6% on a new person, same features, same model, is the size of the **session-memorisation effect**. Treat 57.8% as honest and 93.6% as unusable. Being the person who says that out loud is worth more than any accuracy figure.

Mechanism — why the middle of the band wins [V], `sc_report.html`

The amplitude profile across the 52 subcarriers is a smooth **15.3 dB arch** peaking at band centre, and it is the same arch in both rooms ($r \approx 0.9$). Room multipath would be jagged and room-specific; a smooth reproducible arch is the **radio's own passband roll-off**. Band-edge subcarriers arrive ~15 dB weaker before anyone moves. Directly: SC29 and SC3 record the same walk with the same energy in the 0.5–25 Hz body-motion band — SC3 is *not* blind to the motion, it simply carries ~6 dB more noise underneath it.

The one-line answer: *"Selecting subcarriers here is selecting SNR, not selecting sensitivity."*

Three more facts that make you look prepared [V]

- **Redundancy.** Correlation between subcarriers falls below 0.5 only after a lag of **4**, so the 52 carry roughly **13 independent looks**. During motion the effective rank drops to **3.5–4.2**. That is a measured challenge to the implicit "52 subcarriers = 52× the data" claim.
- **Spreading the picks makes it worse.** At k=8: forcing 4 subcarriers of spacing → 44%, uniform decimation → 45%, against **57.8%** for the clustered high-SNR set. The SNR given up at the edges costs more than decorrelation buys.
- **Index → frequency.** Core eight sit at offsets −6…−3 and +2…+5 × 312.5 kHz — roughly **−1.9 to +1.6 MHz** around 2.462 GHz.

Where the confusion actually is [V], confusion matrix

Walking **87%** correct (234/268). Mixed activity mostly collapses into walking — defensible, since it *contains* walking. Sitting and standing are only partly separable from each other, unsurprising as both are near-static. Collapsed to binary action/no-action: **83.2%** on an unseen person.

Slide 15 (p15) — Active vs Passive CSI Sensing

What is shown. Passive: phone and laptop are the real TX/RX pair; USRP transmits nothing; CSI overheard from existing traffic; traffic uncontrolled and app-dependent. Active: one USRP transmits 802.11 frames, second receives and extracts CSI; channel, rate and timing fully controlled. Photographs of both room setups.

This is Slide 8 restated as a concept. If running long, compress it: *"Same content as the evolution slide, expressed as paradigms."*

One subtlety worth reserving. [S] In the "active" setup the receiver is still passive in the 802.11 sense — it does not associate, ACK, or transmit. It decodes frames off the air like a monitor-mode sniffer. "Active sensing" means *we control the illumination*, not that the receiving side became a network participant.

Numbering bug. [V] This frame and the subcarrier frame are **both numbered 12.** If someone says "go back to slide 12", they mean the subcarrier figure.

Slide 16 (p16) — Data Collection: Walking & Standing

What is shown. Left: measured CSI amplitude trace. Right: photo of the activity. Walking — action take, Amrith, Room A. Standing — no-action take, Rishiikesh, Room A.

What to point at. Do not over-sell; the point is visible.

- **Walking** — large, sustained, broadband excursions across all subcarriers. The whole body moves through the Fresnel zones between the antennas, so path lengths change continuously and by a lot.
- **Standing** — comparatively flat, slow drift, small ripple. That residual is postural sway, breathing and noise. It is not zero, and should not be.

Numbers behind the traces. [V] `pilot_stats.json`

Take	Class	2 s window energy
walk	action	0.01744
mixed	action	0.01138
stand	no-action	0.00364
sit	no-action	0.00326

Aggregated: action **0.01452** vs no-action **0.00346** — a **4.2× energy ratio**, Fisher ratio **3.85**.

Volunteer this before someone else finds it. [V] That separation is large enough that **a single scalar with one threshold classifies 99.1% of windows correctly**. So the binary task is near-trivial on this data — which is exactly why the four-class problem is the one worth reporting.

File naming. [V] `P01_<action|noaction>_<subject>_<date>_<take>.csv`. P01 = pilot 01; the action/noaction field is the binary class; the take field names the activity. Walking and mixed are action; standing and sitting are no-action.

The collection protocol. [V] `PILOT_ACTION_VS_NOACTION.md`: 4 takes of 90 s per subject; walking means crossing the TX–RX line each pass at ~1 step/second, never stopping; mixed is a continuous loop of squat → bend → drink-motion → side-steps, ~13–15 s per loop.

CRITICAL — where this data came from. [M] These traces are from **Setup 1**, the single-USRP passive sniffer, not the two-USRP system. See Section 13 for exactly how to say this.

Slide 17 (p17) — Data Collection: Sitting & Mixed Activity

What is shown. Sitting — no-action, Rishiikesh, Room A. Mixed activity — action, Gowreesh, Room B.

Point out the room. The mixed panel is the only **Room B** example anywhere in the deck, so it is doing double duty as your evidence that the setup was exercised in a second environment. Say *"note this take is from a different room and a different subject"* — otherwise the panel has no visual reason to believe the two-room claim.

Room B is your honest complication — own it. [V] Gowreesh is measurably the hardest hold-out: **32%** at eight subcarriers vs **86%** for Amrith. Concrete reasons:

- His captures needed **8–11%** of samples de-spiked, against 0.2–6.5% in Room A — a noisier link.
- His **static sitting** capture carries as much 0.5–4 Hz band power as Room A's **motion** captures. Something was moving, or the link was worse.
- His session was four days later, so equipment or placement drift is folded in.

If asked "does this work in a different room?":

> "Partly, and we can be specific about which parts. The amplitude profile and the motion-SNR ranking transfer — rho above 0.9 across rooms. The per-activity separability rankings don't transfer at all. And our second room was measurably noisier: it needed eight to eleven percent of samples de-spiked against under seven in the first. So the honest answer is the subcarrier *selection* transfers and the trained *classifier* doesn't yet — which is why per-room calibration is in our next phase."

Slide 18 (p18) — Limitations and Future Work

What is shown. Limitations: channel hopping (sequential, not simultaneous); synchronization (independent SDRs need stable timing/frequency); dataset (large labelled HAR/sign dataset still required). Next phase: CSI pre-processing + features; HAR/sign recognition; CSI + wearable fusion. Long-term: sign-language recognition with identity-aware multi-modal sensing.

As written these are three unquantified bullets. Attach a measured number to each and they become findings. This is where you gain the most ground in the whole review.

Limitation 1 — hopping [V] + [M]

Configuration	CSI rate	Median gap	Largest gap
Parked, single channel	**51–81 Hz**	11–14 ms	0.46 s
Hopping 1/6/11, 100 ms dwell	**1.96 Hz**	129 ms	**55.6 s**

Two internal documents measure this independently and agree in kind: `DATA_COLLECTION.md` reports 1.94 Hz vs 54.9 Hz with ~60 ms settling per 100 ms dwell; `THREE_CHANNEL_ANALYSIS.md` reports 2.4 Hz vs 73.0 Hz with ~110 ms settling. My own re-measurement of the full 2,394 s capture gives **1.96 Hz [M]**. Quote the range honestly — *"between about 2 and 2.4 hertz depending on which capture you read"* — it is stronger than a single suspiciously precise figure.

Three independent reasons ~2 Hz fails. [V]

1. **Nyquist.** Human motion has energy from ~1 Hz gait to tens of Hz limb Doppler. At 2 Hz you do not undersample it — you miss it and alias the remainder into noise.
2. **The gaps outrun the events.** A fall lasts ~1 s; the 95th-percentile gap is ~1 s and the worst observed is 55.6 s. Whole events land inside a gap.
3. **Hopping breaks continuity anyway.** The three channels have different multipath, so the streams cannot be concatenated; each retune resets LO phase, making phase-derived features unusable across hops.

The decisive number: [V] of 854 two-second windows in the hopping capture, **0 (0.0%) contained enough samples to process**, against 93.5% and 88.9% for the two parked captures.

The fix — lead with this:

> "Don't hop during an activity — hop between sessions. Record the full activity set parked on channel 1, then repeat parked on 6, then on 11. You get three complete 50-plus-hertz datasets instead of one unusable 2 Hz interleaved stream, the three-channel comparison that motivates the project survives intact, and the only thing you lose is simultaneity — which activity recognition doesn't need."

Limitation 2 — synchronization [V]

No shared clock. Two hop loops free-run and coincide ~1/3 of the time. No common LO also means residual CFO/SFO on every packet, **which is the reason the analysis is amplitude-only**. Fix: GPSDO per unit, or external 10 MHz + PPS distribution.

Limitation 3 — dataset scale [V]

Dataset	Samples	Note
Ours (pilot)	1,093 windows	3 subjects, 2 rooms, one session each
ARIL	1,394	Same front-end class: USRP N210, 52 subcarriers, SISO, amplitude-only

Dataset	Samples	Note
UT-HAR	4,477	random-split evaluation
Widar 3.0	~258,000	multiple synchronized links

The sharper structural problem underneath the count: each activity is **one continuous capture per person**, so activity is perfectly confounded with recording session. Any slow channel drift is available to the classifier as a shortcut. That confound is exactly what produces the 93.6% vs 41.6% gap. Two concrete fixes, both free: **interleave activities within one session**, and **record at least one repeat session per person on a different day**.

Saying this yourself is the single most valuable thing you can do in this review — it demonstrates you understand what your own numbers mean, which is a higher bar than producing them.

Slide 19 (p19) — References

Grouping, in case you are asked why a paper is there. [V]

- **Refs 1–5, 11** (CASAS, ARAS, UCI public HAR dataset, Opportunity challenge, UniMiB SHAR, Minor et al.) — classical smart-home and wearable HAR datasets. They establish HAR as a mature field and justify the problem statement's claim that existing systems rely on cameras and wearables.
- **Refs 6, 9, 10** (Widar3.0, SignFi, WiFinger) — the WiFi-sensing lineage, from finger gestures through sign language to cross-domain gesture recognition. Your direct line of work.
- **Refs 7, 8** (CSI-Bench 2025, WiMANS 2024) — recent benchmarks, and the most useful two on the slide: they are about **multi-user** sensing and **standardised evaluation**, precisely where your dataset limitation lives. If asked *"what would you evaluate against next?"*, name these.
- **Implementation basis** — GNU Radio, UHD, IEEE 802.11 PHY/MAC processing, custom CSI extraction.

Fix before submission. [V] ImgFi is discussed on the literature slide and is the basis of your own replication, but it is **not in this numbered list**. Add it.

Slide 20 (p20) — Thank You

Notice what is missing: the deck never states dataset size or a single accuracy figure. Slide 14 shows rankings but no validated performance. That will read as an accidental omission if the first accuracy number of the session comes from a panel member's question rather than from you.

Closing line if the deck is frozen:

> "To put numbers on where we are — twelve captures, three people, two rooms, about eighteen minutes of CSI. On a person the model has never seen, eight selected subcarriers give 58% four-class accuracy against 42% using all 52, and 83% on the binary action-versus-no-action question. Within a single capture the same model reaches 94%, and the gap between those two figures is the honest measure of how much work is left. Thank you — happy to take questions."

---

8. Detailed Technical Concepts (taught, then connected to the project)

Only concepts this project actually uses. Each has: simple → technical → why *we* need it → example → viva Q&A.

8.1 OFDM

Simple. Instead of sending one fast stream on one carrier, split the data across many slow streams on many closely spaced carriers sent at once.

Technical. [S] 802.11a/g uses a 64-point IFFT over 20 MHz, giving **312.5 kHz** subcarrier spacing. 48 bins carry data, 4 carry pilots, 1 is DC (unused), 11 are guard bands. A cyclic prefix (16 samples, 0.8 µs) longer than the channel delay spread turns linear convolution into circular convolution, so each subcarrier experiences a *single complex multiplication* by the channel — one number, H_k.

Why we need it. That last property is the whole reason WiFi sensing works. Because each subcarrier sees a flat channel, the receiver must estimate one complex gain per subcarrier — and estimating those 52 numbers *is* measuring the channel. **CSI is free; it falls out of demodulation.**

Example. Bin 6 → offset −26 → 2462 − 8.125 = **2453.875 MHz**. Bin 58 → offset +26 → **2470.125 MHz**.

Q: "Why does OFDM give you 52 measurements instead of 1?" A: Because the cyclic prefix makes each subcarrier's channel a single scalar, and the receiver has to estimate all 52 of them to equalise. We read out that vector. A single-carrier receiver would only give us one number.

8.2 Multipath and frequency-selective fading

Simple. The signal takes many routes to the antenna and they add up; move something in the room and the sum changes.

Technical. [S] The channel is the sum over paths, `H(f) = Σᵢ aᵢ · exp(−j2πf·τᵢ)`, where `aᵢ` is a path's amplitude and `τᵢ` its delay. Two paths whose delay difference is `τ` cancel at frequencies spaced `1/τ` apart, producing periodic nulls across the band. The **coherence bandwidth** — the frequency span over which the channel looks flat — is roughly `1/(5·στ)` for RMS delay spread `στ`; indoors `στ ≈ 50 ns`, giving a coherence bandwidth of a few MHz.

Why we need it. Two consequences drive the project. (1) Nulls can hide motion — that is the frequency-diversity argument for channels 1/6/11, 50 MHz apart, far beyond coherence bandwidth. (2) Within one 20 MHz channel the coherence bandwidth is a few MHz, i.e. ~10 subcarriers — **which is exactly why we measured correlation staying above 0.5 out to a lag of 4 subcarriers and only ~13 independent looks in 52. [V]** The theory and our measurement agree.

Q: "Why are adjacent subcarriers correlated?" A: They're 312.5 kHz apart, and the coherence bandwidth of the room is a few megahertz. So neighbours see essentially the same channel. We measured the correlation dropping below 0.5 only after 4 subcarriers.

8.3 Channel estimation from the LTF

Simple. Every WiFi frame starts with a pattern both sides already know. Compare what arrived to what should have arrived, and the difference is the channel.

Technical. [V] Least squares, in `lib/equalizer/ls.cc`:

$$H[i] = (LTF_1[i] + LTF_2[i]) / (2 \cdot L[i])$$

Averaging the two LTF repetitions gives ~3 dB noise reduction; their difference gives the SNR estimate. Data symbols are equalised as `symbols[c] = in[i] / d_H[i]`.

Alternatives available in the flowgraph. [V] `chan_est` selects **LS (0), LMS (1), Linear Comb (2), STA (3)** — literally radio buttons in the GUI.

Why LS. [L] LS is unbiased and assumes nothing about channel structure. LMS and STA smooth across symbols or subcarriers, reducing noise but also suppressing exactly the fine per-subcarrier variation we want to sense. For communication that trade is good; for sensing it destroys signal.

Q: "Isn't LS the noisiest estimator?" A: Yes, and that's deliberate. The smoothing estimators reduce variance by averaging over subcarriers or symbols — but the per-subcarrier differences they average away are our signal. We accept a noisier estimate to keep the structure intact. It's a one-line change if we want to test the others; they're already exposed in the flowgraph.

8.4 Sampling, Nyquist and Doppler

Simple. You must sample at least twice as fast as the fastest thing you want to see.

Technical. [S] Nyquist: $f_s > 2\, f_{max}$. Motion produces a Doppler shift $f_d = 2\, f_c\, v / c$ for radial velocity v (factor 2 because the path is out-and-back).

The number for this project. [M] At $f_c = 2.462$ GHz: $2 \times 2.462 \times 10^9 / 3 \times 10^8 =$ **16.4 Hz per m/s**. A torso at ~1 m/s → ~16 Hz; a hand at 2 m/s → ~33 Hz.

Our measured position. [V] `comparative_analysis.tex`: *"All class-discriminative information lives below 18 Hz."* Worst-case Nyquist across the pilot captures is 25.7 Hz. So the parked rate is sufficient **for this task** — measured, not assumed.

But quote the caveat honestly. [V] The same document warns this holds because `walk` and `mixed` are *gross* motion. Fine gestures (a finger, a sign) sit higher in frequency and **have not been proven capturable at 66 Hz**.

Q: "How do you know 66 Hz is enough?" A: We measured where the class-discriminative energy lives — it's all below 18 Hz, and our worst-case Nyquist is 25.7 Hz. But that's for walking and gross motion. For sign language we have not proven it, and that's an open item.

8.5 The AUC / separability metric

Simple. How well does one number, on its own, tell class A from class B?

Technical. [V] + [S] `sc_select.py` computes the Mann-Whitney AUC — the probability a random positive sample scores above a random negative one — then maps it to `2·|AUC − 0.5|`. AUC 0.5 (chance) → score 0; AUC 1.0 *or* 0.0 (perfectly separating either way round) → score 1. Score is taken as the **max over the 7 features** for that subcarrier.

Why AUC and not accuracy or t-test. [L]

- Threshold-free: no arbitrary decision boundary to pick.

- Non-parametric: makes no Gaussian assumption, and CSI features are not Gaussian.
- Class-imbalance robust: our four activities have unequal window counts.

Q: "Why not just use a t-test or feature importance from the model?" A: A t-test assumes normality and only tests a mean difference. Model feature-importance would entangle the ranking with the classifier's inductive bias. AUC is threshold-free, distribution-free, and measures exactly what we want: can this one number separate this class?

8.6 mRMR — minimum redundancy, maximum relevance

Simple. Pick features that are useful *and* that don't duplicate each other.

Technical. [V] `sc_select.py::mrmr` — greedy: normalise relevance to [0,1], seed with the single most relevant, then repeatedly add the candidate maximising `relevance[i] − λ · max|corr(i, already_selected)|`, with λ = 0.3.

Why redundancy penalty matters here. [V] Adjacent subcarriers are near-duplicates. Pure top-k on relevance would pick a cluster of eight neighbours all reporting the same thing. mRMR spreads the picks just enough.

But — the measured nuance, and it is counter-intuitive. [V] Forcing *more* spread makes it worse: λ = 0.0 and λ = 0.3 both give 57.8% at k=8, λ = 1.5 gives 54.3%, hard spacing-4 gives 44.4%, uniform decimation 44.5%. **The SNR you give up at the band edges costs more than decorrelation buys.**

Q: "Why λ = 0.3?" A: We tested it — λ from 0 to 1.5 plus four hard-spacing strategies are all in `strategy_cmp` in `sc_scores.json`. 0 and 0.3 tie at the best value; beyond that it degrades. So 0.3 is experimentally chosen, and honestly the redundancy penalty barely matters at k=8 — the relevance term is doing the work.

8.7 Leave-one-subject-out validation

Simple. Train on some people, test on a person the model has never seen.

Why it is the *only* honest protocol here. [V] SignFi drops 98% → 67% on unseen users. **[V]** Our own data shows the same collapse: 93.6% within a capture, 41.6% across people with all 52 subcarriers.

Why a random split is meaningless on this data. [V] `CONTEXT.md` states the rule enforced in `train_imgfi.py`: *"splits are by SUBJECT or filename substring, NEVER random. All 52 images of one window come from the same 2 seconds; a random shuffle leaks near-duplicates across train/test and produces a meaningless ~99%. The script refuses to run without one of those flags."* **That refusal-to-run is a genuinely good piece of engineering and worth mentioning.**

Q: "Why is your accuracy only 58%? Papers report 98%." A: Different protocol. Within a single capture we also get 94%. The 58% is on a person the model has never seen. SignFi itself reports 98% in the lab and 67% on unseen users — the gap is the domain-shift problem, and we chose to report the hard number.

8.8 Recurrence plots

Simple. Turn a wiggly line into a picture, so an image classifier can look at it.

Technical. [V] `recurrence_plot()` builds $RT[i,j] = |x_i - x_j|$ — an $N \times N$ distance matrix from one subcarrier's amplitude series, min-max normalised, resized to 224×224.

Why per-subcarrier and what it costs. [V] `CONTEXT.md`: one image *per subcarrier* → **the dataset becomes 52× larger**; ImgFi reports this as the best-performing and cheapest of its five conversion methods. Prediction is by **majority vote over the 52 images of a window**, with a confidence = votes/52.

The honest caveat you must volunteer. [V] The 52 images of one window come from the same 2 seconds and the subcarriers are highly correlated — effective rank ~4. So the "52× more data" is largely illusory, and `comparative_analysis.tex` says so explicitly. The majority vote is still useful as noise averaging; the dataset-size claim is not.

Q: "Doesn't making 52 images from one window inflate your dataset?" A: Yes, and we measured how much. The subcarriers correlate above 0.5 out to a lag of 4 and the effective rank during motion is about 4 — so there are roughly 13 independent looks, not 52. The vote helps as noise averaging; it doesn't give us 52× the information, and we say that in our analysis.

8.9 De-spiking with MAD

Simple. Throw out impossible jumps before they poison the statistics.

Technical. [V] `sc_features.py`: subtract a 5-tap running median; compute the median absolute deviation; flag residuals beyond `6 × 1.4826 × MAD` and replace with the median. The 1.4826 factor makes MAD a consistent estimator of σ for Gaussian data. **[S]**

Why MAD and not standard deviation. [S] Standard deviation is itself corrupted by the outliers you are trying to find. MAD has a 50% breakdown point.

Why this matters to your results. [V] Room B needed **8–11%** of samples de-spiked vs 0.2–6.5% in Room A — the de-spike rate became a *diagnostic* that quantified how much worse the second environment was.

8.10 Welch's method / PSD

Simple. Estimate how much energy a signal has at each frequency, averaging to reduce noise.

Technical. [V] `scipy.signal.welch` with a Hann window. Per-window: `nperseg = n` (the whole 160-sample window, no averaging). Per-capture: `nperseg=1024, noverlap=512`.

Why detrend first. [V] `signal.detrend(x, type="linear")` before the PSD removes slow gain drift, which would otherwise dump huge power into the lowest bins and swamp the motion bands.

The analysis bands and why they sit where they do. [V] + [S]

Band	Range	Physical meaning
`B_SLOW`	0.15–0.5 Hz	posture sway, breathing
`B_GAIT`	0.5–4 Hz	torso motion, gait fundamental
`B_LIMB`	4–12 Hz	limb swing
`B_FAST`	12–25 Hz	fast limb / hand
`B_NOISE`	**30–38 Hz**	**above any human Doppler → hardware + phase noise**

The noise band is the clever part: it gives a *per-subcarrier, per-window* noise floor measured from the same data, so motion-SNR is a self-referencing quantity requiring no separate calibration.

---

9. Technology Choices and Why — Decision → Reason → Alternative → Trade-off → Result

9.1 USRP N210 as the radio

Decision [V]	Ettus USRP N210, SBX daughterboard, 400–4400 MHz, 40 MHz fixed analog BW, GigE host link
Why needed	To transmit and receive real 802.11 frames with full physical-layer control
Alternatives [S]	Intel 5300 CSI tool (30 subcarriers, no PHY control); Atheros CSI Tool (56 subcarriers, commodity); ESP32-CSI (cheap, 802.11n, limited); PicoScenes
Why chosen [L]	Full PHY control — we write the transmitter, so rate, payload, channel and addressing are ours. That *is* the argument on the motivation slide. Also matches ARIL's front-end spec exactly, giving a like-for-like comparison
Trade-off accepted	Far more expensive, far lower packet rates than commodity tools, and **the 20 MHz / GigE ceiling that blocks simultaneous 3-channel capture**
Result	Controlled acquisition achieved; simultaneous multi-channel proven infeasible on this hardware **[V]**

Honest note: the repository does not record a written trade study choosing N210 over commodity tools. The N210 was likely already available in the lab. Present the reasoning as *"the control it gives us is what the project needs"* — do not claim you evaluated alternatives.

9.2 GNU Radio + `gr-ieee802-11`

Decision [V]	Bastian Bloessl's `gr-ieee802-11` OOT module on GNU Radio 3.10.11.0, one block modified
Why needed	Implementing an 802.11a/g PHY from scratch is a multi-year project
Alternatives [S]	Write our own PHY; use `srsRAN` / `openwifi` (FPGA-based, different hardware); MATLAB WLAN Toolbox (no real-time SDR loop)
Why chosen [L]	It is the reference open-source 802.11 SDR implementation, it already exposes the equaliser's channel estimate internally, and it runs on the N210 through UHD
Trade-off	We inherit its bugs and its metadata conventions — **which is exactly what bit us**: `parse_mac` emits `ta` / `ra` only for Control frames, and our Data frames were silently dropped **[V]**

Result	A working PHY in weeks instead of years; one deep debugging episode

9.3 BPSK rate 1/2 modulation

Decision [V]: `encoding = ieee802_11.BPSK_1_2`. **Why [L]:** It is the most robust 802.11a/g rate — lowest required SNR, so the highest fraction of transmitted frames decode successfully. **For sensing, throughput is irrelevant and decode probability is everything**, because a frame that fails its FCS produces no CSI row at all. Choosing the most robust modulation directly maximises the CSI sampling rate. **Alternative:** 64-QAM 3/4 would carry ~10× the data at the same packet rate, but needs ~20 dB more SNR and would drop frames at range. **Trade-off:** none that matters — we are not carrying user data. The payload is `SEQ=…;TS=…;LABEL=…` padding.

9.4 200 packets per second

Decision [V]: `PPS = int(os.environ.get("PPS", 200))`, raised from 50. **Repository's own stated reason, verbatim [V]:** *"PPS is the ceiling on the receiver's CSI sampling rate — the RX can never log more rows/s than the TX sends packets/s, and it will log fewer once frame errors are counted. 200 leaves headroom above the ~50 Hz that activity recognition needs."* **This is a verified, documented justification — use it. If increased:** more headroom, but higher channel occupancy and more interference with real WiFi; at some point the PHY pipeline or the host cannot keep up. **If decreased:** at 50 pps the ceiling equals the requirement, so any frame loss immediately pushes you below the ~50 Hz target. That is why it was raised.

9.5 Amplitude-only features

Decision [V]: `amp = np.hypot(re, im)`, then dB. Phase discarded. **Reason [V]:** `CONTEXT.md` — *"amplitude only — raw phase is unusable, the USRP isn't synced to the transmitter."* **Technical principle [S]:** measured phase = true phase + CFO·t + SFO·k + packet-detection delay + LO phase noise. With independent oscillators these terms are unknown per packet and larger than the Doppler being measured. **Alternatives [S]:** linear detrend of phase across subcarriers (removes SFO and delay, not CFO); conjugate multiplication between two RX antennas (needs 2 chains); a shared 10 MHz/GPSDO reference. **Trade-off:** we lose the more informative half of the signal. **Result:** consistent with ARIL, which is amplitude-only on the same front-end class. **Every published amplitude-only result is achievable; Doppler-based methods like Widar3.0's BVP are not.**

9.6 Two-second windows

Decision [V]: `WIN_S = 2.0`, `HOP_S = 1.0` (50% overlap) in `sc_features.py`; `WINDOW_SECONDS = 2` in the notebook; 52 × 200 in `segment_windows.py`. **Why 2 s [L] + [V]:** long enough to contain a full gait cycle (~1 s) plus margin, and long enough for meaningful frequency resolution — at 2 s the PSD bin width is 0.5 Hz, which is just enough to separate the 0.5–4 Hz gait band from the 4–12 Hz limb band. **[V]** The pilot protocol's mixed-activity loop is 13–15 s, so a 2 s window sits inside a single sub-motion. **If longer (4 s):** better frequency resolution, but fewer windows (already only 1,093) and more chance of two different motions inside one label. **If shorter (1 s):** more windows, but 1 Hz bin width can no longer resolve the gait band, and a 1 s window may not contain a complete gait cycle. **Why 50% overlap [S]:** standard practice — with a Hann window, 50% overlap gives near-uniform weighting of all samples across windows. It also doubles the window count, which matters on a small dataset. **Caveat you should volunteer: overlapping windows are not independent samples, so they inflate apparent dataset size.**

9.7 Random Forest for the subcarrier validation

Decision [V]: `RandomForestClassifier(n_estimators=200, min_samples_leaf=2)`, 400 trees for the final confusion matrix. **Why [L]:** the validation question is *"does this subcarrier subset carry the information?"*, not *"what is the best possible classifier?"* A random forest needs no feature scaling, handles the 7 heterogeneous features (dB, Hz, dB/sample) without normalisation, is nearly hyperparameter-free, and does not overfit catastrophically on 1,093 samples the way a deep net would. **It keeps the measurement about the features rather than about tuning. Alternative:** an end-to-end CNN — but then a change in accuracy could not be attributed to subcarrier selection versus architecture. **Result:** a clean, attributable comparison across k = 1…52.

9.8 ResNet18 in the ImgFi notebook — the choice you must be careful about

Decision [V]: `models.resnet18(weights=ResNet18_Weights.DEFAULT)` — pretrained, images resized to 224×224, grayscale replicated to 3 channels, ImageNet normalisation, Adam lr = 1e-4, batch 64, 15 epochs, CrossEntropyLoss, ReduceLROnPlateau. **The tension [V]:** the deck's literature slide praises ImgFi for being **~90× smaller than ResNet50**. ResNet18 is ~11.7 M parameters — smaller than ResNet50 but far from ImgFi's lightweight net. The 44.8 MB checkpoint confirms it. **Likely rationale [L]:** with ~300 windows, ImageNet initialisation converges reliably where a small net trained from scratch may not. **How to say it:** see Slide 7 above. **Do not claim a lightweight-model result. Note [V]:** `csi_tools/train_imgfi.py` *is* the paper-faithful 3-conv CNN. Both exist; be clear about which produced the number you quote.

10. Algorithms and Methodology

10.1 Acquisition algorithm (per capture)

1. Park TX and RX on the same channel (`TX_CHANNEL` / `CSI_CHANNEL`), or start both hop loops.
2. Set `CSI_TA` / `CSI_RA` to the synthetic MAC pair; set `CSI_OUT` .
3. Start `wifi_rx.py` , then `wifi_tx_multi_hop.py` , then `packet_generator.py` .
4. Watch for `(MATCH)` and a `rows/s` figure.
5. `capture.sh` records one labelled instance and **warns if the rate is under 20 Hz** — *"do not build a dataset on this"*. [V]

10.2 Feature extraction (`sc_features.py`) [V]

1. Load CSV; drop rows with NaN; sort by timestamp; normalise time to start at 0.
2. `amp = hypot(Re, Im)` ; `amp_db = 20·log10(max(amp, 1e-12))` .
3. De-spike: 5-tap running median, MAD, 6σ threshold.
4. Linear-interpolate each subcarrier onto a uniform **80 Hz** grid.
5. Record `gap_frac` — fraction of time inside inter-arrival gaps longer than 2 grid steps — as a data-quality metric.
6. Cut 2 s windows, hop 1 s.
7. Per window: linear detrend, Welch PSD, then compute **7 features** per subcarrier:

Feature	Definition	Units
`sigma`	std of detrended dB series	dB
`roughness`	median absolute first difference	dB/sample
`p_gait`	10·log10 power in 0.5–4 Hz	dB
`p_limb`	10·log10 power in 4–12 Hz	dB
`p_fast`	10·log10 power in 12–25 Hz	dB
`msnr`	10·log10((gait+limb+fast) / noise-band)	dB
`centroid`	spectral centroid over 0.5–25 Hz	Hz

Design principle worth quoting [V]: *"deliberately no absolute level, so nothing can separate classes by per-capture gain alone."* Every feature is a *dynamics* feature. If you had included mean amplitude, the classifier could separate captures by receiver gain drift and score beautifully while learning nothing about motion.

Output shape: `X` = (1093 windows, 7 features, 52 subcarriers).

10.3 Subcarrier scoring and selection (`sc_select.py`) [V]

Two questions, two metrics, because they behave differently — this is the intellectual core of the work.

Metric 1 — per-activity separability. One-vs-rest, `2·|AUC − 0.5|` maxed over the 7 features, computed **per subject**, averaged within Room A, then combined as the **element-wise minimum** of Room A and Room B ("worst case of the two rooms"). *Reliable within a capture, does not transfer.*

Metric 2 — motion-SNR profile. The `msnr` feature, median over each capture's windows, then **z-scored inside each capture** so only the *shape across frequency* survives. Averaged over the 6 action captures; the *worst case* over those 6 is used as relevance. *This transfers.*

The split between them is the whole result. [V] Quoting `sc_report.html` : *"A subcarrier's ability to separate this sitting capture from this standing capture is a fact about one afternoon's multipath. How strongly a subcarrier responds to a body moving is a fact about the radio, and it is stable."*

Then: correlation matrix over per-capture-standardised `sigma` ; coherence lag = first lag where mean correlation < 0.5 → **4**, giving 52/4 = **13** independent looks. mRMR with λ=0.3 over the worst-case motion-SNR relevance → the order `29, 21, 28, 30, 23, 20, 22, 27, 31, 19, 14, 26, …` . First 8 sorted = **SC20–23, 27–30**.

Then validate: random forest, leave-one-subject-out, k = 1, 2, 4, 6, 8, 10, 12, 16, 20, 26, 52, each compared against a random set of the same size (10 draws), the *worst* k subcarriers, and time-blocked within-capture CV.

10.4 The ImgFi classification pipeline [V]

`CSV → segment_windows.py (52×200 amplitude .npy) → recurrence plot per subcarrier (RT[i,j]=|xi−xj|) → 224×224 grayscale → ResNet18 → per-image logits → majority vote over 52 images → per-window prediction`. Split by subject via `create_split(test_subject)`. Per-window test accuracy 95.9–97.8% on the binary task **[V, notebook outputs]**.

---

11. Important Parameters — every number, explained

#	Parameter	Value	Where [V]	Why this value	If ↑	If ↓	Type
1	Carrier, ch 11	2462 MHz	`sc_features.py` `FC`; Setup 1 config	Only 3 non-overlapping 2.4 GHz channels; 11 is the top one	ch 11 is already the top	ch 1 = 2412 MHz, more crowded	Standard
2	Channels used	1 / 6 / 11	`CHANNELS` dict	Only non-overlapping trio in the band	—	overlapping channels aren't independent	Standard
3	Sample rate	20 MS/s	both flowgraphs	Matches the 20 MHz 802.11 channel; **also the GigE ceiling**	GigE caps ~25 MS/s	would clip the channel	HW-constrained
4	RX bandwidth	20 MHz	`set_bandwidth(20e6)`	One WiFi channel	SBX filter fixed at 40 MHz	loses signal	HW
5	FFT size	64	802.11a/g	Standard	—	—	Standard
6	Subcarrier spacing	312.5 kHz	20 MHz / 64	Standard	—	—	Standard
7	Subcarriers kept	52	`get_csi()` bins 6–58 excl. 32	48 data + 4 pilots; rest are guard/DC with no signal	can't — no signal there	discards information	Standard
8	RX gain	50 dB	`set_gain(50, 0)`	Near top of range, maximise SNR on a weak indoor link	front-end saturation, clipping	lower SNR, more decode failures	Empirical [L]
9	TX gain	20 dB	`set_gain(tx_gain)`	Rooms are 3–5 m; avoid saturating RX and interfering with real WiFi	RX saturation, interference	frames fail FCS → fewer CSI rows	Empirical [L]
10	Modulation	BPSK 1/2	`encoding`	Most robust rate → highest decode probability → highest CSI rate	fewer frames decode	already the lowest	Reasoned [L]
11	Packet rate	200 pps	`packet_generator.py`	**Documented:** ceiling on CSI rate; leaves headroom above the ~50 Hz needed	more occupancy/interference	at 50 pps any loss drops below target	**Documented [V]**
12	Payload	500 bytes	`PAYLOAD_SIZE`	Enough for `SEQ` / `TS` / `LABEL`; matches `packet_length=500` in TX	longer frames, lower pps ceiling	header overhead dominates	Arbitrary-ish [L]
13	Dwell time	100 ms	`DWELL_TIME = 0.1`	**Initial guess — not documented.** Measured to be ~40% productive	500 ms ≈ 88% productive	50 ms: settling exceeds dwell, nothing gets through	**Arbitrary, then measured [V]**
14	`sync_short` threshold	0.56	`sync_short(0.56, 2, …)`	Upstream default	more missed frames	more false detections	Upstream default
15	`sync_long` window	320 samples	`sync_long(320, …)`	Upstream default	—	—	Upstream default
16	Channel estimator	LS (`chan_est=0`)	`wifi_rx.py`	Unbiased, no smoothing that would erase per-subcarrier sensing signal	LMS/STA smooth away signal	—	Reasoned [L]

#	Parameter	Value	Where [V]	Why this value	If ↑	If ↓	Type
17	MAC pair	`02:…:01` → `02:…:02`	`ieee802_11.mac(...)`	`02:` = locally administered, guaranteed no collision with real hardware	—	—	Standard [S]
18	Resample grid	80 Hz	`sc_features.py` `FS`	Above every capture's native rate (51–81 Hz) so all are on one grid	more interpolation artefact	would alias down the faster captures	Reasoned [V]
19	Window length	2 s	`WIN_S`	Contains a gait cycle; gives 0.5 Hz PSD resolution	fewer windows, mixed labels	can't resolve the gait band	Reasoned [L]
20	Window hop	1 s	`HOP_S`	50% overlap, standard for Hann	fewer windows	more correlated, non-independent windows	Standard
21	Noise band	30–38 Hz	`B_NOISE`	Above any human Doppler (16.4 Hz per m/s) → pure hardware noise	would exceed Nyquist on slow captures	would include real limb motion	**Physically motivated [V]**
22	De-spike threshold	6 × 1.4826 × MAD	`sc_features.py`	1.4826 makes MAD ≈ σ for Gaussian; 6σ is conservative	misses real spikes	starts removing genuine fast motion	Standard [S]
23	Coherence lag	4 subcarriers	**measured**, `coh_lag`	First lag where mean correlation < 0.5	—	—	**Measured [V]**
24	Independent looks	13	52 / 4	Derived from above	—	—	**Derived [V]**
25	Recommended k	8 (extend 12)	`curve` peak	Peak of the LOSO accuracy curve	12→55.5%, 20→54.1%, 52→41.6%	4→54.8%, 1→41.5%	**Measured [V]**
26	mRMR λ	0.3	`mrmr(..., lam=0.3)`	Tested 0/0.3/0.6/1.0/1.5; 0 and 0.3 tie best	1.5 → 54.3%	—	**Measured [V]**
27	RF trees	200 (400 final)	`n_estimators`	Enough for a stable estimate on 1,093 samples	diminishing returns	higher variance	Standard
28	RF min leaf	2	`min_samples_leaf=2`	Mild regularisation on a small dataset	underfit	overfit	Standard
29	Notebook rate	50 Hz	`RATE = 50`	Below every capture's native rate, so no upsampling	—	—	Reasoned [L]
30	Image size	224 × 224	`transforms.Resize`	ResNet18's expected ImageNet input	—	pretrained weights mismatch	Constrained by model
31	Learning rate	1e-4	`Adam(lr=1e-4)`	Standard for fine-tuning a pretrained net	unstable fine-tuning	slow convergence	Standard [S]
32	Batch size	64	`DataLoader`	Fits GPU memory at 224×224	memory	noisier gradients	Practical
33	Epochs	15	`EPOCHS = 15`	Train accuracy hits 1.000 by epoch ~6 **[V, outputs]**	more overfitting	underfit	Empirical
34	Segmentation rate	100 Hz default	`segment_windows.py --rate`	**This is the bug** — see §13	—	`--rate 50` fixes it	**Wrong default [V]**

The `--rate` bug, stated properly. [V] `segment_all_pilot.py` ran with the default `--rate 100` against captures whose native rate is ~66 Hz. A 2-second window then requires 200 samples where only ~132 exist, so windows below the threshold were skipped: **48 of 370 windows (13%) silently dropped.** `analysis/README.md` records the fix: pass `--rate 50`. **No recapture needed.** If asked, this is a good answer — you found it, quantified it, and it is fixable in a flag.

---

12. Code / Repository Mapping — "show me where this is implemented"

PPT claim	File	Symbol / line	What it does
"Controlled IEEE 802.11 transmission"	`Three_Channel_Experiment/transmitter/gr-ieee802-11/examples/wifi_tx_multi_hop.py`	`wifi_tx.__init__`, `ieee802_11.mac(...)`, `wifi_phy_hier`	Builds and transmits frames
"Packet generation"	`…/examples/packet_generator.py`	main loop	200 pps, 500-byte payload over TCP 52001
"Sequential hopping, 100 ms dwell"	both flowgraphs	`CHANNELS`, `DWELL_TIME`, `hop_channels()`	Daemon thread retunes and labels
"USRP N210 RX / IQ capture"	`receiver/…/wifi_rx.py`	`uhd_usrp_source_0`	addr 192.168.10.7, RX2, 20 MS/s, gain 50
"802.11 Sync — Short + Long"	`receiver/…/wifi_rx.py`	`sync_short(0.56, 2, …)`, `sync_long(320, …)`	Packet detection, fine sync
"FFT and frame equalization"	`receiver/…/wifi_rx.py`	`fft_vxx_0`, `frame_equalizer(Equalizer(chan_est), …)`	64-pt FFT, LS equalisation
"CSI Extraction" — where CSI is born	`receiver/gr-ieee802-11/lib/equalizer/ls.cc`	`ls::equalize`, `n==1` branch	`d_H[i] = (LTF1+LTF2)/(2·L[i])`
"52 subcarriers"	`…/lib/equalizer/base.cc`	`base::get_csi()`	Keeps bins 6–58 except 32
"Custom CSI extraction"	`…/lib/extract_csi_impl.cc`	`handle_pdu()`	MAC filter, channel stamp, CSV row
"Channel label 1/6/11"	same	`set_channel()`, `d_current_channel`	Written as column 3
"CSI samples arranged over time"	`USRP_lib/csi_tools/segment_windows.py`	`--rate`, `--window`	CSV → 52×200 `.npy`
Slide 12 figure	`analysis/sc_figures.py`	`sc_fig4_top_per_activity`	The four bar charts
Slide 12 numbers	`analysis/sc_features.py` → `sc_select.py` → `sc_scores.json`		Features, scoring, LOSO validation
Slides 16–17 traces	`ppt_images/graph_*.png`	built from `data/pilot/`	Amplitude panels
Energy / Fisher numbers	`analysis/analyze_pilot.py` → `pilot_stats.json`	`separability`	4.2×, Fisher 3.85, 99.1%
Hopping feasibility verdict	`Three_Channel_Experiment/doc/analyze_channels.py` → `channel_stats.json`; `THREE_CHANNEL_ANALYSIS.md`		1.94–2.4 Hz, 0% usable windows
Labelled recording	`receiver/capture.sh`		Env setup, 20 Hz gate, clean shutdown
Classifier	`Action_NoAction_imgfi.ipynb`	`ImqFiDataset`, `create_split`, `majority_vote`	RP images → ResNet18 → per-window vote
Alternative classifier	`USRP_lib/Action_NoAction_CNN_GRU.ipynb`, `csi_tools/train_hybrid.py`	`SpatialTemporalAttentionModel`	Conv1d ×2 → BiGRU ×2 → attention → FC

The CNN-GRU-Attention architecture, if asked. [V] Input (batch, 52, 200) → `Conv1d(52→64, k=5, pad=2)` + BN + ReLU + `MaxPool(2)` → `Conv1d(64→128, k=5, pad=2)` + BN + ReLU + `MaxPool(2)` → (batch, 128, 50) → `GRU(input=128, hidden=128, layers=2, bidirectional)` → (batch, 50, 256) → attention `Linear(256→1)` over time → weighted sum → `Linear(256→64)` + ReLU + `Dropout(0.5)` + `Linear(64→2)`. **Design logic [L]:** the Conv1d treats the 52 subcarriers as *channels* and time as the spatial axis, so it learns cross-subcarrier patterns at each instant; the

BiGRU models temporal evolution in both directions; attention lets the model weight the informative part of the 2-second window rather than averaging over it.

---

13. PPT vs Actual Implementation — read this section twice

13.1 Shown in the PPT and correctly implemented ✔

Claim	Verified by
Two-USRP TX–RX architecture	Both flowgraphs exist and run
Controlled IEEE 802.11 transmission	`mac` + `wifi_phy_hier` + `packet_generator.py`
Channels 1 / 6 / 11 at 2412 / 2437 / 2462 MHz	`CHANNELS` dict, both sides
≈100 ms dwell	`DWELL_TIME = 0.1`, both sides
Sequential channel hopping	`hop_channels()` threads; capture contains all three channels **[M]**
Channel-labelled CSI	`Channel` column in `extract_csi_impl.cc`
52-subcarrier complex CSI	`get_csi()` returns 52
802.11 sync → FFT → equalization → CSI extraction	The literal flowgraph
Per-activity subcarrier rankings	`sc_scores.json → top_per_activity` matches the slide exactly
Does NOT receive 1/6/11 simultaneously	Correctly stated; §6.3 of the analysis explains why

13.2 🔴 CRITICAL — shown in the PPT, but the data came from a different system

The deck's narrative implies the CSI data on pages 14, 16 and 17 came from the two-USRP setup. It did not.

	Evidence
Claim implied	Pages 8–13 build the two-USRP setup; pages 14–17 then show CSI data with no statement that the setup changed
Reality [M]	All 12 `data/pilot/*.csv` lack the `Channel` column that Setup 2 always writes → they are **Setup-1 passive captures**
Corroborating [V]	`comparative_analysis.tex` §2 describes the pilot setup as *"USRP N210 passively sniffs an existing MacBook → Android-hotspot [link]"*
What Setup 2 has actually produced [M]	4 hopping tests (0, 9, 14 and 4,688 rows) and 4 parked attempts (**all 0 rows**)
Do you need to know this for the viva?	**Yes. This is the highest-risk item in the entire deck.**

How to say it — turn it into a strength:

> "I should be explicit about which system produced this data. The CSI traces and the subcarrier analysis come from our earlier single-USRP passive setup, parked on channel 11 — that's the dataset we have eighteen minutes of across three people and two rooms. The two-USRP system is built and verified end-to-end: we see matched frames and all three channels labelled correctly. What it hasn't produced yet is a HAR dataset, because when we measured the hopping mode it delivered about two hertz of CSI against the fifty-plus we need. So this phase's deliverable is the controlled acquisition system plus the measurement that told us how to use it — record parked, one channel per session — and that recapture is the immediate next step."

Why this is genuinely defensible: you built the system, you measured it, the measurement produced an actionable design change. That is engineering. What is *not* defensible is implying the pilot data came from it.

13.3 🟠 Implemented but NOT shown in the PPT

Thing	Where	Should you mention it?
Full subcarrier-selection validation — 58% vs 42%, LOSO	`sc_scores.json`, `sc_report.html`	**Yes — your best result and the deck omits it entirely**

Thing	Where	Should you mention it?
The hopping feasibility study with numbers	`THREE_CHANNEL_ANALYSIS.md`, `DATA_COLLECTION.md`	**Yes — makes the limitations slide quantitative**
The `ta` / `ra` → `address 2` / `address 1` bug fix	`extract_csi_impl.cc`	Yes, if asked what was hard
Parked-mode env switches (`TX_CHANNEL` / `CSI_CHANNEL`)	both flowgraphs	Yes — it is the recommended operating mode
`capture.sh` with a 20 Hz quality gate	`receiver/capture.sh`	Yes if asked about data quality control
The classifiers themselves (ResNet18; CNN-GRU-Attention)	notebooks, `csi_tools/`	Yes if asked "what model?"
Comparison against ARIL / UT-HAR / Widar3.0	`comparative_analysis.pdf`	Yes — good answer to "how does this compare?"
Effective rank ≈ 4, 13 independent looks	`pilot_stats.json`, `sc_scores.json`	Yes — strong, non-obvious measurement
`check_capture.py` QA tooling	`csi_tools/`	Only if asked

13.4 🟠 In the PPT but implemented differently

PPT says	Reality [V]	Impact
"Amplitude **and** phase provide the sensing information" (p12)	**Amplitude only.** Phase discarded — no shared clock	Medium. Have the CFO/SFO answer ready
Caption on p14: ranked, filtered across rooms/subjects, thinned, validated on unseen person	Describes the **recommended set** (SC20–23, 27–30). The **figure** shows per-activity rankings, which the same analysis shows do **not** transfer	**High.** Section 7 gives the script
ImgFi is "lightweight, ~90× fewer params than ResNet50" (p7)	Your notebook uses **pretrained ResNet18** (~11.7 M params, 44.8 MB checkpoint). The lightweight version exists in `csi_tools/train_imgfi.py` but is not what produced the quoted number	Medium. Be clear which implementation
"Sequential hopping" presented as the working mode	Hopping measured at ~2 Hz and judged unusable; the project's own recommendation is to **park**	**High.** Own it on the limitations slide

13.5 🟡 In the PPT but not clearly supported

Claim	Status
"Exploit frequency diversity to **study and improve** HAR robustness" (p3)	**No frequency-diversity benefit has been measured.** No experiment compares single-channel against multi-channel HAR accuracy. It is a motivation, not a result. **Do not claim improvement.**
"HAR: use temporal CSI variations for activity classification" listed as *completed* (p5)	True, but on Setup-1 data and offline. Say so
"Privacy-preserving"	Directionally true; CSI has no imagery. But [S] CSI can support keystroke/gait inference — say "less intrusive than a camera", not "private"

13.6 Simplifications in the PPT (fine, but know the detail)

- "802.11 Sync — Short + Long" hides autocorrelation-based detection, AGC, coarse and fine CFO estimation.
- "Frame Equalization" hides the choice among LS/LMS/Linear-Comb/STA.
- "CSI Extraction" hides the MAC filter, the append-mode CSV, per-row flushing, and the `(MATCH)` diagnostics.
- The block diagram omits that the last connection is a **message port**, not a stream.

13.7 Fix list before submitting the deck

1. **Two frames numbered 12** (subcarrier selection; active-vs-passive). Renumber → frames 1–17.
2. **Two frames numbered 4** — deliberate continuation; label the second "4 (cont.)".
3. **ImgFi missing from the reference list** though cited on the literature slide.
4. **Page 14 caption describes a different result than the figure shows** — either reword, or add the recommended set SC20–23, 27–30.
5. **Page 14 has no axis labels explained on the slide** — add "y: separability 0–1 · x: subcarrier index 0–51".
6. **No results slide anywhere** — dataset size and accuracy appear nowhere in 20 pages.
7. **State the data provenance** on page 16 — e.g. a footnote "pilot dataset, single-USRP parked capture, channel 11".

8. `analysis/README.md` **is stale** — it says "2 subjects, 1 room, 322 samples"; current reality is 3 subjects, 2 rooms, 1,093 windows. Do not circulate it alongside the talk.

---

14. Results and Graph Analysis

14.1 The CSI amplitude traces (slides 16–17)

Plotted: CSI amplitude (linear magnitude `|H|`, unitless — a ratio of received to transmitted amplitude) on y, against time in seconds on x. **52 coloured lines**, one per subcarrier.

Trend and why. [S] Walking gives large, sustained, broadband excursions — the torso crosses the Fresnel zones between the antennas, changing many path lengths by many wavelengths per second. Standing and sitting give a comparatively flat trace with slow drift and small ripple: residual sway, breathing, and receiver noise.

What it proves: motion produces a channel perturbation far larger than the static-case fluctuation — 4.2× in window energy. [V]

What it does NOT prove: that *different* activities are separable from each other. Standing vs sitting are both near-static; our own confusion matrix shows they are only partly separable. **Do not let a trace comparison stand in for a classification result.**

"Why is there noise on the flat trace?" Three sources: thermal noise in the receiver front end; residual phase noise leaking into the amplitude estimate; and genuine micro-motion — a standing human sways and breathes. [S] The 30–38 Hz noise band in our feature set exists precisely to quantify the first two.

"How do you know the trace is correct?" `check_capture.py` performs the QA: rows/s, gap distribution, NaN / zero / constant columns, and an amplitude-vs-time plot for the "wiggle test" — you wave a hand during capture and confirm the trace moves. [V] Every pilot capture has a `_check.png` alongside it in `data/pilot/`.

14.2 The subcarrier separability bar charts (slide 14)

Plotted: y = robust separability, dimensionless, range 0–1, defined as $2 \cdot |AUC - 0.5|$ maxed over 7 features and then min-ed across the two rooms. x = subcarrier index, 0–51. Top ten highlighted.

Trend. Each activity peaks in a different region: walking at SC26–28 and SC36–39; standing low at SC4–8 and SC18–23; sitting at the extremes SC2–5 and SC40–44; mixed around SC33–39 and SC45–48.

Why the trend occurs. Two effects superimpose: [V] the smooth 15.3 dB passband arch (the radio's own roll-off, reproducible across rooms at $r \approx 0.9$) sets the SNR available at each subcarrier, and on top of it sits room- and session-specific multipath structure. **The arch transfers; the structure does not.**

What it proves: subcarriers are not interchangeable; some carry substantially more usable motion information than others.

What it does NOT prove — and this is the honest limit: that *these specific rankings* generalise. Cross-subject correlation of the ranking within the same room is **negative** (−0.14 to −0.43). [V] The plot is an accurate measurement of the captures we have, not a deployable configuration.

Is it statistically meaningful? Partly, and we tested exactly that. [V] Split-half reliability within a capture is $\rho = 0.72$–0.91, so the measurement is repeatable. The failure is not noise — it is that the quantity being measured is session-specific.

14.3 The k-vs-accuracy validation curve

Plotted: leave-one-subject-out 4-class accuracy (y, %) against number of selected subcarriers k (x, log-ish scale $1 \rightarrow 52$), with four series: selected set, random set of the same size, worst set, and within-capture CV.

Trend. Selected rises steeply to a **peak at k=8 (57.8%)**, then declines gently and collapses at k=52 (41.6%). Random hovers flat at ~41–44%. Worst sits at ~35%. Within-capture rises monotonically to 93.6%.

Why: adding subcarriers adds both information and noise. Because there are only ~13 independent looks, information saturates by about k=8, while the noise from low-SNR band-edge subcarriers keeps accumulating. Past the peak you are adding noise dimensions to a 1,093-sample problem.

What it proves: subcarrier selection is not cosmetic — **it is worth +16 accuracy points** over using everything, and the selected set beats a random set of equal size at every $k \geq 2$, which rules out "any 8 would do".

What it does NOT prove: that k=8 is optimal in general. [V] `sc_report.html` says it plainly: *"It is the best-supported choice available, not a proven optimum."* With 3 subjects, the difference between 57.8% at k=8 and 55.5% at k=12 is within the noise of a 3-fold hold-out.

Chance level: 4 classes → 25%. So 57.8% is 2.3× chance, and 41.6% (all 52) is still above chance. Say this — it frames the number correctly.

14.4 The confusion matrix [V]

Rows = true, columns = predicted, order walk / mixed / stand / sit:

	→ walk	→ mixed	→ stand	→ sit	recall
walk	**234**	33	0	1	**87.3%**
mixed	168	**90**	16	18	30.9%
stand	49	21	**155**	40	58.3%
sit	54	25	55	**134**	50.0%

Read it out loud like this: walking is solid at 87%. Mixed activity mostly collapses into walking — which is *reasonable*, because the mixed take contains side-steps, so it literally contains walking. Standing and sitting confuse with each other and with walking, which is expected for near-static classes. Collapsed to binary action/no-action the same predictions give **83.2%** on an unseen person.

The one honest weakness to name first: mixed's 30.9% recall. Do not hide it — explain it as label overlap, not model failure.

14.5 Honest results summary — what to claim and what not to

Result	Value	Protocol	Claim strength
Action vs no-action energy ratio	4.2×, Fisher 3.85	descriptive	**Solid**
Single scalar + threshold, binary	99.1%	within-dataset	**Solid — and it shows the binary task is trivial**
ImgFi/ResNet18 binary	95.9–97.8% per window	leave-one-subject-out, **2 subjects, 1 room**	**Real, but on a near-trivial task, and it does not beat the threshold**
CNN-GRU-Attention binary	98.4–99.3%	LOSO per window	Same caveat [V] `comparative_analysis.tex` : *"It does not beat the single-feature baseline"*
4-class, 8 subcarriers	**57.8%**	LOSO, 3 subjects, 2 rooms	**The honest headline**
4-class, all 52	41.6%	same	Solid
4-class within capture	93.6%	time-blocked CV	**Report only as the memorisation ceiling — never as performance**
Binary, 12 subcarriers	83.2%	LOSO, 3 subjects	Solid
Hopping CSI rate	1.94–2.4 Hz	measured on 2 captures	Solid
Frequency diversity **improves** HAR	—	**not measured**	**Do not claim**

---

15. Limitations — the complete honest list

1. **The two-USRP system has not produced a HAR dataset.** All results are from Setup 1. [M]
2. **Hopping is unusable at ~2 Hz** — 0 of 854 windows processable. [V]
3. **No clock synchronization** between the radios; hop loops free-run and coincide ~1/3 of the time. [V]
4. **Amplitude only** — phase discarded for lack of a shared reference. [V]
5. **Simultaneous 3-channel reception is blocked by hardware** — 70 MHz needed, 20 MHz available over GigE, 40 MHz fixed SBX filter, and the transmitter has the same limit. [V]
6. **Dataset scale** — 1,093 windows vs ARIL 1,394 / UT-HAR 4,477 / Widar3.0 258,000. [V]
7. **Session confound** — one continuous capture per activity per person, so activity is perfectly confounded with recording session. This is *why* within-capture reaches 93.6%. [V]
8. **Single occupant only.** Every capture has one person. Multi-person is untested. [V]
9. **Room B is measurably noisier** — 8–11% de-spike rate, and its static sitting capture carries motion-band power. [V]
10. **Per-activity subcarrier rankings do not transfer** across subjects or rooms. [V]
11. **Windows overlap 50%**, so window counts overstate independent sample count. [S]
12. **13% of windows were silently dropped** by a wrong `--rate` default. Fixable in a flag, no recapture. [V]
13. **Sign language has not been attempted at all.** No sign data exists in the repository. [M]
14. **Fine-gesture capturability at 66 Hz is unproven** — the Nyquist argument covers gross motion only. [V]
15. **Sit/stand are only partly separable** — 58% and 50% recall. [V]

16. **Offline only** — no real-time classification path exists. [V]
17. **"Privacy-preserving" is directional, not absolute** — CSI supports some inference. [S]
18. **No frequency-diversity benefit has been demonstrated**, only motivated. [M]

---

16. Future Work — with the reasoning attached

Immediate (weeks):

1. **Recapture parked, one channel per session.** Run the full activity set on ch 1, then ch 6, then ch 11. Yields three complete ~50 Hz datasets and preserves the three-channel comparison. **[V] — this is the repository's own recommendation.**
2. **Interleave activities within a session** to break the session confound — the single biggest source of inflated accuracy.
3. **Repeat sessions on a different day** per person — the only way to separate a stable subcarrier preference from an afternoon's multipath.
4. **Re-run segmentation with** `--rate 50` to recover the 13% of dropped windows.
5. **Add a results slide** to the deck.

Medium (this phase): 6. **GPSDO or external 10 MHz + PPS** on both USRPs → unlocks phase, which unlocks Doppler-based methods. 7. **Fall detection (RO1)** — a favourable signal: short, high-energy, abrupt. 8. **First sign-language captures (RO3)** — with a rate check first, since fine gestures sit higher in frequency. 9. **Validate the recommended 8 subcarriers on the new parked data** — the current recommendation rests on 3 subjects.

Longer term: 10. **Wearable IMU fusion (RO4)** for identity tagging and fall confirmation. 11. **Evaluate against CSI-Bench / WiMANS** for standardised comparison. 12. **X310 + UBX-160 + 10 GbE** if genuine simultaneous multi-channel is required.

---

17. Questions Outside the PPT — from the repository

Q. What does `capture.sh` do, and why does it exist?

[V] It records one labelled activity instance to its own CSV. It requires `CSI_CHANNEL` to be set to 1/6/11, refuses to overwrite an existing file, forbids underscores in subject/activity names (because loaders split the filename on `_` into exactly three fields, so `sit_stand` would silently mislabel), sets the MAC filter environment, gives a 3-second countdown, runs the flowgraph under `timeout -s INT`, and then **warns if the achieved rate is under 20 Hz** — *"do not build a dataset on this."* **Why `-s INT` specifically:** so the flowgraph takes the same clean-shutdown path as Ctrl+C — `sig_handler` stops the top block and `extract_csi`'s destructor flushes and closes the CSV. A plain SIGTERM would skip that and truncate the file.

Q. Why does `extract_csi` open the CSV in append mode?

[V] So that pointing `CSI_OUT` at an existing file never destroys data. The header is written only if the file is new or empty.

Q. What happens if the MAC filter doesn't match?

[V] Nothing is written, and the block prints each distinct `seen TA=… RA=…` pair once with `(no match)`. That print is the primary live diagnostic. Setting `CSI_TA=any CSI_RA=any` accepts everything and is the recommended first test. **Real incident worth telling:** the teammates' `har_receiver.py` wrapper hardcoded `CSI_TA=42:42:42:42:42:42`, which matches nothing our transmitter sends. It would never have produced a `(MATCH)`. [V]

Q. What happens when the capture rate is too low?

[V] Three layers catch it: `capture.sh` warns below 20 Hz; `segment_windows.py` skips windows with too few samples and prints *"WARNING: more skipped than saved"*; `sc_features.py` records `gap_frac` per capture as a quality metric.

Q. What happens with missing or invalid data?

[V] `sc_features.py` drops rows with NaN in any subcarrier or the timestamp, sorts by timestamp (so out-of-order rows are handled), de-spikes outliers, and linearly interpolates onto a uniform grid — so gaps are bridged rather than crashing. `analyze_pilot.py` separately counts `truncated_rows_dropped` (1 per file — the last row, cut short at shutdown) and `zero_rows`.

Q. What is the computational complexity / where is the bottleneck?

[S] + [V] Acquisition is real-time and the bottleneck is the radio, not compute: 20 MS/s complex over GigE ≈ 160 Mbit/s. Feature extraction is O(N·52·log W) per capture and takes seconds. The random-forest LOSO sweep over 11 values of k with 10 random baselines each is the slowest offline step — minutes. Training ResNet18 on 52 images per window is the only GPU-scale job; that is why the notebook ran on Colab.

Q. How does the system scale?

Honestly: **it does not, yet.** One transmitter, one receiver, one room, one occupant. Scaling to multiple rooms means one TX/RX pair per room (or accepting through-wall attenuation); scaling to multiple occupants is an unsolved research problem, not an engineering one.

Q. Why is there a `Three_Channel_Receiver/` folder as well?

[V] It is the teammates' raw drop — the version they got building — plus an older ZMQ-based attempt and a `USRP_HAR_Project/` wrapper that are now dead ends. `Three_Channel_Experiment/receiver/` carries forward only the current parts. Say "we consolidated the working parts and left the earlier attempt in the tree for history."

Q. Why is `usrp/uhd/` vendored if you don't build it?

[V] `CONTEXT.md`: reference only. The project does not build or modify UHD.

Q. How is the model saved and loaded?

[V] `torch.save` to `imgfi_model_fold1.pth` (44.8 MB zipped at the repo root). Fold 1 means the first leave-one-subject-out fold.

Q. What does `check_capture.py` check?

[V] rows/s, gap distribution, NaN / zero / constant columns, and it renders an amplitude-vs-time PNG. Every pilot capture has a `_check.png` beside it.

Q. Why 52 columns of Re and Im rather than magnitude?

[L] Storing the raw complex value keeps the option open — phase is unusable *today*, but if a shared clock reference is added later, the existing captures cannot be re-derived if only magnitude was stored. Storing complex costs nothing and is irreversible if skipped.

Q. Why is `extract_csi::work()` empty?

[V] Because the block is driven entirely by the `pdu in` message port. It is declared as a `sync_block` with a 52-complex output signature for API compatibility with upstream, but all logic lives in `handle_pdu` and the stream output goes to a null sink in the flowgraph.

---

18. WHY QUESTIONS — HIGH PRIORITY

Why WiFi CSI instead of a camera?

Short: Privacy, and it works in the dark and through furniture. **Detailed:** The rooms where falls happen — bathroom, bedroom — are exactly the rooms nobody accepts a camera in. CSI records 52 complex numbers per packet describing a radio channel; there is no image and no audio. **Technical reasoning:** Human motion perturbs multipath propagation; that perturbation is fully captured in the per-subcarrier channel estimate the receiver already computes. **Follow-up: "But CSI can still identify people, can't it?"** → Yes, under favourable conditions the literature shows gait and even keystroke inference. So the accurate claim is *substantially less intrusive than a camera*, not *private*. We would not claim anonymity.

Why a dedicated SDR transmitter instead of sniffing real traffic?

Short: Because otherwise an app decides our sampling rate. **Detailed:** Passive CSI exists only when someone else sends a packet. A video call gives a torrent; an idle screen gives nothing. Human limb motion has energy into the tens of hertz, so we need a guaranteed rate and roughly uniform spacing. **Technical reasoning:** Nyquist plus the uniform-sampling assumption baked into every CSI tool — STFT, wavelet, sliding-window PCA. **Follow-up: "Did passive actually fail?"** → It didn't fail — we got 51–81 Hz, which is enough for gross motion. But the rate was not *ours*, so it was neither guaranteed nor reproducible. And for finer gestures we'd need to raise it, which passively we cannot.

Why channels 1, 6 and 11?

Short: They're the only three non-overlapping channels in the 2.4 GHz band. **Detailed:** Channel centres are 5 MHz apart but each channel is 20 MHz wide, so 1/6/11 at 2412/2437/2462 MHz are the only trio that don't overlap. 50 MHz separation is far beyond the coherence bandwidth of a room, so their fading is effectively independent. **Follow-up: "Did multi-channel actually improve accuracy?"** → **Honest answer: we have not measured that.** It's the motivation, not a result. To measure it we need three parked datasets, which is exactly what our recapture plan produces.

Why did you park instead of hop, when hopping is the whole use case?

Short: Because we measured hopping and it delivers about 2 Hz. **Detailed:** Retuning costs a fixed amount per hop — LO settling plus OFDM re-acquisition — measured at roughly 60–110 ms against a 100 ms dwell. Of 854 two-second windows in a 40-minute hopping capture, **zero** had enough samples to process. **Technical reasoning:** the cost is per *hop*, not per second, so shortening the dwell doubles the overhead. At 50 ms nothing gets through at all. **Follow-up: "So the multi-channel objective failed?"** → No — it changed shape. We get the same three-channel comparison by recording each channel in its own session. We lose only *simultaneity*, and activity recognition doesn't need it. What we did lose is the ability to study fast channel-to-channel correlation, which would need hardware that retunes 500× faster.

Why amplitude and not phase?

Short: No shared clock between the two radios, so raw phase is a random walk. **Detailed:** Measured phase = true phase + carrier frequency offset × time + sampling frequency offset × subcarrier index + packet-detection delay + LO phase noise. Those terms are unknown per packet and larger than the Doppler we want. **Follow-up: "How would you fix it?"** → Three options: a GPSDO in each USRP or a distributed 10 MHz + PPS reference; a linear detrend of phase across subcarriers, which removes SFO and detection delay but not CFO; or conjugate multiplication between two receive antennas, which needs a second RX chain. The first is what we'd do.

Why least-squares channel estimation?

Short: Because the alternatives smooth away exactly what we're trying to sense. **Detailed:** LMS and STA reduce estimator variance by averaging across subcarriers or symbols. For communication that's good. For sensing, the fine per-subcarrier variation *is* the signal. **Follow-up: "Have you tested the others?"** → Not systematically — they're exposed as radio buttons in our flowgraph so it's a one-line experiment, and it's on the list. Our reasoning for the default is the smoothing argument, not a measured comparison. **[Be honest here.]**

Why 8 subcarriers rather than all 52?

Short: Because 8 gets 58% on an unseen person and 52 gets 42%. **Detailed:** There are only ~13 independent looks among the 52 — correlation stays above 0.5 out to a lag of 4 — and the band-edge subcarriers sit ~15 dB down the radio's passband, so they contribute noise rather than information. Adding them to a 1,093-sample problem adds dimensions without adding signal. **Technical reasoning:** selecting subcarriers is selecting SNR, not sensitivity. SC3 sees the same motion energy as SC29; it just carries 6 dB more noise underneath it. **Follow-up: "Why not spread them across the band for diversity?"** → We tested exactly that. Forcing 4 subcarriers of spacing gives 44%, uniform decimation 45%, against 58% for the clustered high-SNR set. The SNR you give up at the edges costs more than the decorrelation buys.

Why leave-one-subject-out instead of a random split?

Short: Because a random split on this data is meaningless. **Detailed:** Our windows overlap 50%, and in the ImgFi pipeline all 52 images of a window come from the same 2 seconds. A random shuffle puts near-duplicates on both sides of the split. Our own training script **refuses to run** without a subject or filename split. **Technical reasoning:** the quantity of interest is generalisation to a *new person*, which is the deployment condition. **Follow-up: "What does that cost you in headline accuracy?"** → About 36 points on the 4-class task — 93.6% within-capture versus 57.8% cross-person. That gap is the honest measure of how much of the usual CSI accuracy is session memorisation.

Why a random forest for the subcarrier study rather than a neural network?

Short: Because the question is about the features, not the classifier. **Detailed:** A random forest needs no feature scaling, handles our seven heterogeneous features (dB, Hz, dB/sample) directly, is nearly hyperparameter-free, and won't overfit catastrophically on 1,093 samples. That keeps any change in accuracy attributable to the subcarrier subset. **Follow-up: "Would a CNN do better?"** → Probably on within-capture accuracy, and probably not on cross-person, because the bottleneck is dataset diversity rather than model capacity. That's testable and it's on the list.

Why 2-second windows?

Short: Long enough to contain a gait cycle and to resolve the gait band. **Detailed:** A walking gait cycle is about a second, so 2 s contains a full cycle plus margin. And a 2 s window gives 0.5 Hz PSD resolution, which is just enough to separate the 0.5–4 Hz gait band from the 4–12 Hz limb band. Our mixed-activity protocol loop is 13–15 s, so a 2 s window sits inside one sub-motion. **Follow-up: "Why not 4 seconds?"** → Better frequency resolution but half as many windows, and we only have 1,093. It also raises the chance that two different motions fall inside one label.

Why 200 packets per second?

Short: It's the ceiling on our CSI rate, and it leaves headroom above the ~50 Hz we need. **Detailed:** This is documented in the code — the receiver can never log more rows/s than the transmitter sends. It was raised from 50, because at 50 the ceiling equalled the requirement and any frame loss pushed us below target. **Follow-up: "Why not 1000?"** → Higher channel occupancy and more interference with real WiFi in the building, and at some point the PHY chain and the host can't keep up. 200 gives 4× headroom, which is enough.

Why BPSK rate 1/2 — the slowest modulation?

Short: Because for sensing, decode probability matters and throughput doesn't. **Detailed:** A frame that fails its FCS produces no CSI row at all. BPSK 1/2 is the most robust 802.11a/g rate, so the highest fraction of transmitted frames decode successfully — which directly maximises the CSI sampling rate. We're not carrying user data; the payload is a sequence number and a label.

---

19. Difficult Mentor / Viva Questions

"What is the novelty of this project?"

> "The novelty isn't algorithmic — we're using established CSI sensing techniques. Two things are genuinely ours. First, a fully controlled SDR sensing link where every acquisition parameter — rate, channel, payload, modulation, addressing — is a parameter we set rather than something we inherit. Second, a measured negative result: three-channel hopping on a USRP N210 delivers about 2 Hz of CSI, and we can attribute it precisely to per-hop settling. The literature doesn't report this because published multi-channel systems assume radios that retune 500 times faster. That's a useful finding for anyone attempting this on this hardware class."

"What is your actual contribution, versus what you downloaded?"

> "The 802.11 PHY is Bastian Bloessl's `gr-ieee802-11` — we did not write that. Ours is the `extract_csi` block that filters by MAC and writes labelled CSV, the channel column and `set_channel` hook, the hop loops and park modes on both radios, the capture tooling, and the entire analysis pipeline — segmentation, features, subcarrier selection, validation. Also one real bug fix without which nothing worked: the address filter looked only for metadata keys GNU Radio emits for Control frames, and our transmitter sends Data frames, so every frame was being silently dropped."

"What is the biggest weakness of your approach?"

> "Dataset diversity. Three people, two rooms, one session each, and each activity recorded as one continuous block — which means activity is perfectly confounded with recording session. That's why the same model gets 94% inside a capture and 58% on a new person. The fix costs nothing but time: interleave activities within a session and record repeat sessions on different days. Until we do that, I'd treat 58% as the real number and 94% as unusable."

"Your accuracy is 58%. Is that acceptable?"

> "For a four-class problem, chance is 25%, so it's over twice chance on a person the model has never seen — and it beats using all 52 subcarriers by 16 points, which is the actual result we were testing. But no, 58% isn't deployable. The binary action/no-action question, which is what the segmentation use case actually needs, reaches 83%. And I'd rather report 58% honestly than the 94% we could get from a within-capture split."

"What if your assumption about the noise band is wrong?"

> "The assumption is that 30–38 Hz contains no human Doppler. At 2.462 GHz the Doppler is 16.4 Hz per metre per second of radial speed, so 30 Hz corresponds to about 1.8 m/s — faster than a hand moves in normal activity, but not faster than a hard swing. If it's wrong, our motion-SNR is slightly pessimistic for fast gestures. It's applied identically to every capture, so comparisons still hold — but for sign language, which has faster hand motion, we'd need to re-examine it."

"What happens if the network connection fails during capture?"

> "The USRP link is a direct ethernet cable, not a network, so there's no routing to fail. If the cable drops, UHD reports overflows and the flowgraph stops producing samples — the CSV simply ends. Because we flush every row, everything captured up to that instant is on disk. If the TCP connection between `packet_generator.py` and the flowgraph drops, the transmitter stops sending and the receiver's CSV goes quiet, which is visible immediately in the `rows/s` print."

"What if the model predicts wrongly in deployment?"

> "It depends which direction. For the segmentation use case, a false 'signing started' hands a non-sign clip to the recogniser, which then returns low confidence — recoverable. A missed 'signing started' loses a message, which is worse. For fall detection the asymmetry is stronger: a false alarm is an annoyance, a missed fall is a safety failure — so we'd tune the threshold toward recall and pair it with the wearable as a confirmation channel. That's part of the motivation for RO4."

"Why not use cloud instead of processing locally?"

> "The capture side has to be local — it's a real-time 20 MS/s stream into a GNU Radio flowgraph over a direct ethernet link, so there's nothing to offload. Training happened in Colab because of GPU access. For deployment, inference on a 2-second window is small enough to run on the capture machine, and keeping it local is consistent with the privacy argument: if CSI never leaves the house, there's nothing to intercept."

"What if there are two people in the room?"

> "Our system doesn't handle it, and I wouldn't claim otherwise. Every capture we have is single-occupant. CSI is a superposition of all reflections, so two movers produce a mixed signal that our per-window features can't separate. That's a known open problem — WiMANS, one of our references, is a recent benchmark specifically for multi-user WiFi sensing. Our planned answer is the wearable: it tells us *who*, and CSI tells us *what*."

"Show me where the CSI is actually computed."

> "`Three_Channel_Experiment/receiver/gr-ieee802-11/lib/equalizer/ls.cc`, in `ls::equalize`. On the second long training symbol it does `d_H[i] += in[i]; d_H[i] /= LONG[i] * 2` — that's the least-squares estimate `H = (LTF1 + LTF2) / (2·L)`. `base.cc`'s `get_csi()` then copies out FFT bins 6 to 58 excluding 32, which is the 52 used subcarriers. Our block reads it out of the PDU metadata in `extract_csi_impl.cc`."

"Why should I believe your 1.94 Hz number?"

> "It's reproducible from the repository. `data/3ch_test_20260813_001142.csv` has 4,688 rows over 2,394 seconds — that's 1.96 Hz, and the three channels are represented 1,555 / 1,734 / 1,399. Two independent write-ups in the repo measured it separately and got 1.94 and 2.4 Hz on different captures. The parked comparison is 51 to 81 Hz across our twelve pilot captures."

"If you had to justify this project's existence in one sentence?"

> "Because the two groups this serves need continuous sensing in exactly the rooms where a camera is unacceptable and a wearable gets left on the nightstand — and WiFi is already in the room."

---

20. "If Mentor Asks This, Say This" — spoken form

Mentor: *So this data is from your new two-USRP setup?* **Me:** "No — I should be clear about that. This is from our earlier single-USRP passive setup, parked on channel 11. The two-USRP system is built and verified, but when we measured its hopping mode it gave about two hertz of CSI, so we haven't recaptured the dataset on it yet." **If she asks "Why not?" →** "Because at two hertz, zero out of 854 two-second windows had enough samples to process. Recapturing would have given us a worse dataset than the one we already had. The fix is to park on one channel per session, and that recapture is the immediate next step." **If she pushes "Then what did you achieve this phase?" →** "A controlled acquisition system where rate, channel and addressing are ours instead of inherited, and the measurement that tells us how to operate it. Also a real bug fix — the address filter was dropping every frame silently — without which the system produced nothing at all."

Mentor: *Explain your subcarrier selection.* **Me:** "We asked two different questions with two different metrics. First, which subcarriers separate one activity from the others — that's a one-vs-rest AUC score. Second, which subcarriers actually respond to motion relative to their own noise floor. The first is highly repeatable inside a capture but doesn't survive a change of person. The second does — rho above 0.8 across people and 0.9 across rooms. So we built the recommendation only from the second, de-duplicated it because neighbouring subcarriers are near-copies, and validated it on a person the model had never seen." **If she asks "What did you find?" →** "Eight subcarriers — 20 to 23 and 27 to 30 — give 58% four-class accuracy on an unseen person, against 42% using all 52. Cutting the input by 85% buys accuracy rather than costing it." **If she asks "Why does that happen?" →** "There are only about 13 independent looks among the 52, because neighbours correlate above 0.5 out to a lag of 4. And the band edges sit 15 dB down the radio's own passband, so they carry noise rather than signal. Selecting subcarriers is selecting SNR, not sensitivity."

Mentor: *Why is your accuracy so low compared to the papers?* **Me:** "Different evaluation protocol. Within a single capture we also reach 94%. Our 58% is on a person the model has never seen. SignFi, which is in our literature survey, reports 98% in the lab and 67% on unseen users — the same collapse." **If she asks "So which number is real?" →** "58%. The 94% is measuring how well the model memorised which recording a window came from, because each activity is one continuous capture — activity is perfectly confounded with session." **If she asks "How do you fix that?" →** "Interleave the activities inside one session instead of recording one block each, and record a repeat session per person on a different day. Both cost only time."

Mentor: *Does phase matter?* **Me:** "It carries more information than amplitude in principle, and it's what Doppler-based methods like Widar3.0 depend on. We don't use it, because our two radios have no common clock." **If she asks "What exactly goes wrong?" →** "Every packet's measured phase includes an unknown carrier frequency offset, a sampling frequency offset and a packet-detection delay. Those are larger than the Doppler we're trying to measure, so packet to packet the phase is close to a random walk." **If she asks "How would you get it?" →** "A GPSDO in each USRP, or a distributed 10 MHz and PPS reference. That's the single hardware change that would unlock the most capability for us."

Mentor: *What's the sampling rate and is it enough?* **Me:** "51 to 81 hertz across our twelve captures, about 66 on average. We didn't assume it was enough — we measured where the class-discriminative energy actually is, and it's all below 18 hertz, against a worst-case Nyquist of 25.7." **If she asks "Why 18 Hz?"** → "Doppler at 2.462 gigahertz is 16.4 hertz per metre per second of radial speed, so a torso at walking pace lands near 16. That's the physical reason the energy stops where it does." **If she asks "Will it be enough for sign language?"** → "Unproven, and I wouldn't claim it. Hand and finger motion is faster, so it sits higher in frequency. Checking that is the first step of RO3."

21. General Technical Fundamentals — quick reference

Concept	One-line version	Where it bites this project
Nyquist	Sample at >2× the highest frequency	2 Hz hopping vs ~16 Hz torso Doppler — the whole hopping verdict
Doppler shift	$f_d = 2 f_c v / c$	16.4 Hz per m/s at 2.462 GHz; sets every analysis band
Wavelength	$\lambda = c/f$ = 12.2 cm here	A 6 cm displacement flips a reflected path 180° — why CSI is so motion-sensitive
Multipath	Signal arrives by many routes and they sum	The sensing mechanism itself
Coherence bandwidth	Frequency span over which the channel looks flat	A few MHz indoors → adjacent subcarriers correlate → only ~13 independent looks
OFDM	Many slow narrowband carriers instead of one fast one	Gives 52 channel estimates for free
Cyclic prefix	Guard that makes convolution circular	Why each subcarrier's channel is a single complex number
CFO / SFO	Oscillator mismatch between TX and RX	Why phase is unusable without a shared clock
AGC	Automatic gain control	Part of why absolute amplitude level is untrustworthy — hence dynamics-only features
LS estimation	Divide received by known transmitted	Literally how CSI is computed here
FCS	Frame check sequence (CRC)	A failed FCS means no CSI row — why robust modulation raises the sampling rate
AUC	P(random positive ranks above random negative)	The separability metric
mRMR	Pick relevant, non-redundant features	The selection algorithm
MAD	Robust spread estimate, 50% breakdown	The de-spiker
Welch PSD	Averaged periodogram	Every band-power feature
Spectral centroid	Frequency "centre of mass"	Distinguishes fast from slow motion
Effective rank	How many dimensions actually carry variance	~4 during motion, vs 52 nominal
LOSO	Hold out a whole person	The only honest protocol here
Confusion matrix	Per-class error structure	Shows mixed collapsing into walk
Random forest	Bagged decision trees	The validation classifier
Transfer learning	Reuse ImageNet weights	What the ResNet18 notebook actually does
Recurrence plot	$RT[i,j] = \lvert x_i - x_j \rvert$	1-D series → image for a 2-D CNN
Locally administered MAC	`02:` prefix = not vendor-assigned	Why our synthetic MACs can't collide

22. "Show Me the Code" — instant answers

If she says...	Open this	And say
"Where is CSI computed?"	`receiver/gr-ieee802-11/lib/equalizer/ls.cc`	"`ls::equalize`, the `n==1` branch — LS estimate from the two LTF symbols"
"Where do the 52 come from?"	`lib/equalizer/base.cc`	"`get_csi()` — FFT bins 6 to 58, excluding DC bin 32"

If she says...	Open this	And say
"Where is your custom block?"	`lib/extract_csi_impl.cc`	" `handle_pdu` — MAC filter, channel stamp, CSV row, flushed per row"
"Where is the MAC bug fix?"	same file, `handle_pdu`	"Falls back to `address 2` / `address 1` when `ta` / `ra` are absent — Data frames vs Control frames"
"Where is the hopping?"	`wifi_rx.py` / `wifi_tx_multi_hop.py`	" `hop_channels()` in a daemon thread; `CHANNELS` and `DWELL_TIME` at the top"
"Where do you set the packet rate?"	`packet_generator.py`	" `PPS` , default 200, environment-overridable"
"Where are the features?"	`analysis/sc_features.py`	" `window_features()` — seven dynamics features per subcarrier"
"Where is the selection?"	`analysis/sc_select.py`	" `sep_scores()` , then `mrmr()` , then `loso_acc()` "
"Where are the results?"	`analysis/sc_scores.json`	" `curve` , `confusion` , `reliability` , `mrmr_order` "
"Where is the model?"	`Action_NoAction_imgfi.ipynb`	"ResNet18 on recurrence plots, split by `create_split(test_subject)` "
"Where is the hopping verdict?"	`Three_Channel_Experiment/DATA_COLLECTION.md` §0	"The rate table and the three reasons 2 Hz fails"
"Where is the bandwidth argument?"	`doc/THREE_CHANNEL_ANALYSIS.md` §6.3	"70 MHz needed, 20 MHz over GigE, 40 MHz SBX filter"
"Where is data quality checked?"	`csi_tools/check_capture.py` , `receiver/capture.sh`	"Rate, gaps, NaN/zero columns; and a 20 Hz gate in the capture script"

23. Elevator Explanations

30 seconds

> "We're building activity recognition for a smart home using WiFi channel state information instead of cameras, for users who sign and users who can't hear. This phase we replaced a passive sniffer with a controlled two-USRP link — we now own the transmitter, so packet rate and channel are parameters instead of accidents. We also measured that three-channel hopping delivers about 2 Hz of CSI against the 50-plus that activity recognition needs, so we record each channel separately instead."

1 minute

> "When a WiFi signal travels across a room it arrives by many paths — direct, plus reflections off walls and off the human body. Those reflections change as a person moves, and the receiver already measures that as channel state information: 52 complex numbers per packet. That's our sensor. No camera, no wearable. Our target users are speech-impaired people, who sign, and hearing-impaired people, who need household events surfaced visually. The hard problem isn't recognising a sign — it's knowing when signing has started, because raising a hand to sign and raising a hand to clean look similar. So the first thing we built is activity-versus-stillness segmentation. This phase we moved from passively sniffing someone else's WiFi link to a dedicated two-USRP transmitter and receiver, which means we control the packet rate, the channel and the timing. We implemented hopping across channels 1, 6 and 11, then measured it — and found it delivers about 2 Hz, which is unusable. So our recommendation is to record each channel parked in its own session. And separately, we showed that 8 of the 52 subcarriers beat all 52 on a person the model has never seen: 58% versus 42% on a four-class problem."

3 minutes

> Start with the 1-minute version, then continue: "On the physics — at 2.4 GHz the wavelength is 12 centimetres, so a body moving a few centimetres changes a reflected path by a large fraction of a cycle. That's why CSI is so motion-sensitive. And the Doppler shift is 16.4 hertz per metre per second, so a torso at walking pace puts energy around 16 hertz. We measured that all our class-discriminative energy sits below 18 hertz against a worst-case Nyquist of 25.7 — so our sampling rate is sufficient for gross motion, and we say 'gross motion' deliberately because we haven't proven it for fine gestures. On the implementation — we build on GNU Radio and Bastian Bloessl's 802.11 module. We didn't write the PHY. What's ours is the CSI extraction block that filters by MAC address and writes labelled CSV, the channel labelling, the hopping control on both radios, and the whole analysis pipeline. One thing worth mentioning: the address filter originally looked only for metadata keys that GNU Radio emits for Control frames, but our transmitter sends Data frames — so every frame was being silently discarded and the pipeline produced an empty file while appearing to run perfectly. On results — our dataset is 12 captures, three people, two rooms, about 18 minutes, 1,093 two-second windows. The honest headline is 58% four-class on an unseen person with 8 subcarriers, and 83% on the binary action-versus-no-action question. Within a single capture the same model gets 94% — and that gap is the honest measure of how much of the usual CSI accuracy is session memorisation, because each activity is one continuous recording so activity is confounded with session. On limitations — no clock synchronization between the radios, so we use amplitude only; simultaneous three-channel reception is blocked by bandwidth, since we'd need 70 megahertz and gigabit ethernet gives us 20; and the dataset is small. Next steps are recapturing parked one channel at a time, interleaving activities within a session to break the confound, and adding a GPSDO so phase becomes usable."

---

24. Top 50 Likely Viva Questions

🔴 Very likely

1. Explain your entire project. → §23
2. What is CSI? → 52 complex channel estimates per packet, from the 802.11 long training field
3. Why WiFi instead of a camera? → §18
4. What changed this phase? → passive sniffer → controlled two-USRP link
5. Why channels 1, 6 and 11? → only non-overlapping trio
6. What is the packet/sampling rate? → 200 pps TX; 51–81 Hz CSI parked; 1.96 Hz hopping
7. Why is your accuracy only 58%? → LOSO vs within-capture; §19
8. How are the two USRPs synchronized? → **they are not**; §7 slide 9
9. Do you use amplitude or phase? → amplitude only; why
10. Why 52 subcarriers? → 48 data + 4 pilots; guard bands carry no signal
11. Where does CSI actually come from in the code? → `ls.cc`, LS estimate
12. What is your dataset? → 12 captures, 3 people, 2 rooms, 18.6 min, 1,093 windows
13. What are the limitations? → §15
14. What is the contribution vs downloaded code? → §19
15. Why does hopping not work? → per-hop settling; 0 of 854 windows usable

🟠 Likely

16. Why 100 ms dwell? → initial guess, then measured ~40% productive
17. Why 2-second windows? → gait cycle + 0.5 Hz PSD resolution
18. Why leave-one-subject-out? → §18
19. What is the confusion structure? → walk 87%, mixed collapses into walk
20. Why 8 subcarriers? → 58% vs 42%; only ~13 independent looks
21. Why not spread the subcarriers across the band? → tested: 44–45% vs 58%
22. Which model did you use? → ResNet18 on recurrence plots; also CNN-GRU-Attention
23. What is a recurrence plot? → $RT[i,j] = |x_i - x_j|$
24. Why BPSK 1/2? → decode probability, not throughput
25. Why 200 pps? → documented: ceiling on CSI rate, 4× headroom
26. What is the Doppler at this frequency? → 16.4 Hz per m/s
27. How do you know the sampling rate is enough? → energy < 18 Hz, Nyquist 25.7 Hz
28. Why least squares? → smoothing estimators erase the sensing signal
29. Can you receive all three channels at once? → no: 70 MHz needed, 20 available
30. What happens if the MAC filter is wrong? → nothing written; `(no match)` printed
31. How did you label the data? → filename convention; `capture.sh` ; `session_logger.py`

32. What preprocessing do you do? → de-spike, dB, resample to 80 Hz, detrend, window
33. Why de-spike with MAD? → std is corrupted by the outliers you're hunting
34. What is the noise band and why 30–38 Hz? → above any human Doppler
35. How many people can it track? → one; multi-person is unsolved
36. Is it really privacy-preserving? → less intrusive than a camera; not zero-information
37. What is the effective dimensionality? → ~4 during motion, ~13 independent looks
38. How does it compare to ARIL / UT-HAR / Widar3.0? → `comparative_analysis.pdf`
39. What is the novelty? → §19
40. What would you do differently? → interleave activities; repeat sessions; GPSDO

🟡 Possible

41. What is the wavelength / subcarrier spacing? → 12.2 cm / 312.5 kHz
42. Which FFT bins map to which subcarriers? → 6–58 excluding 32
43. Why RX2 antenna? → receive-only port, no T/R switch
44. Why gain 50 / gain 20? → SNR vs saturation; **[L]**, no documented sweep
45. Why environment variables for configuration? → GRC regeneration would overwrite arguments
46. What does `capture.sh` protect against? → overwrite, underscores in labels, sub-20 Hz rate
47. What is the computational bottleneck? → the 20 MS/s radio link; then RF LOSO sweep
48. What happens on a dropped connection? → CSV ends cleanly; flushed per row
49. Why is `work()` empty in your block? → PDU message port drives everything
50. What is `Three_Channel_Receiver/`? → teammates' raw drop; older ZMQ attempt, dead end

---

25. Things I MUST Memorize

Identity. Smart Home HAR for Speech- & Hearing-Impaired Users · 23CSE399 Phase 2 · Type 2 Application Project · Guide Ms. Nalinadevi K.

Problem. Cameras fail on privacy, wearables fail on compliance, uncontrolled WiFi fails on measurement quality. A controlled CSI link clears all three.

Objective. Controlled, privacy-preserving CSI acquisition + HAR; RO1 falls, RO2 subcarriers, RO3 sign language, RO4 multimodal. **RO2 is partly done.**

Architecture. `packet_generator (200 pps) → mac → wifi_phy_hier (BPSK 1/2) → USRP N210 TX` — RF — `USRP N210 RX → sync_short → sync_long → FFT(64) → frame_equalizer (LS) → decode_mac → parse_mac → extract_csi → CSV`.

Where CSI is born. `ls.cc`: `H[i] = (LTF1[i] + LTF2[i]) / (2·L[i])`. `base.cc get_csi()`: bins 6–58 excluding 32 = 52.

Numbers — the non-negotiable list.

Value	Meaning
52	subcarriers (48 data + 4 pilots)
2412 / 2437 / 2462 MHz	channels 1 / 6 / 11
20 MHz, 20 MS/s, 312.5 kHz	channel width, sample rate, subcarrier spacing
12.2 cm	wavelength at 2.462 GHz
16.4 Hz per m/s	Doppler sensitivity
200 pps	transmit rate
100 ms	dwell time
51–81 Hz	CSI rate parked (mean ~66)
1.94–2.4 Hz	CSI rate hopping
0 of 854	usable windows while hopping
12 / 3 / 2 / 18.6 min / 1,093	captures / people / rooms / duration / windows
2 s window, 1 s hop, 80 Hz grid	segmentation
7	features per subcarrier per window
4.2× , Fisher 3.85	action vs no-action energy
99.1%	single scalar + threshold, binary

Value	Meaning
SC20–23, 27–30	the recommended eight
57.8% / 41.6%	4-class LOSO at k=8 / k=52
93.6%	4-class within capture (memorisation ceiling)
83.2%	binary LOSO at k=12
87%	walking recall
4 / 13	coherence lag / independent looks
15.3 dB	passband arch
< 18 Hz / 25.7 Hz	discriminative energy / worst-case Nyquist
70 / 20 / 40 MHz	span needed / GigE limit / SBX filter

File names to have on the tip of your tongue. `extract_csi_impl.cc` · `ls.cc` · `base.cc` · `wifi_rx.py` · `wifi_tx_multi_hop.py` · `packet_generator.py` · `capture.sh` · `sc_features.py` · `sc_select.py` · `sc_scores.json` · `pilot_stats.json` · `DATA_COLLECTION.md` · `THREE_CHANNEL_ANALYSIS.md` · `Action_NoAction_imgfi.ipynb`

Terminology. CSI · OFDM · subcarrier · LTF/STF · least squares · CFO/SFO · multipath · coherence bandwidth · Fresnel zone · Doppler · Nyquist · AUC · mRMR · MAD · Welch PSD · spectral centroid · effective rank · LOSO · recurrence plot · UHD · GNU Radio OOT module · PDU/message port.

Team contributions (as identifiable from the repository — verify with your team before stating). The receiver-side channel-aware `gr-ieee802-11` is credited in `Three_Channel_Experiment/README.md` to teammates; the transmitter packaging, hop control, capture tooling and the entire `analysis/` pipeline are the current author's. Pilot data subjects: Amrith and Rishiikesh (Room A, 17 July), Gowreesh (Room B, 21 July). **[V]**

---

26. Final Viva Cheat Sheet

Open with (if asked "explain your project"): the 1-minute version in §23.

Three sentences you should get out no matter what:

1. *"We own the transmitter now, so sampling rate is a parameter rather than an accident."*
2. *"Cutting 52 subcarriers to 8 doesn't cost accuracy, it buys it — 58% against 42% on an unseen person."*
3. *"Within a capture we get 94%, across people 58%. The gap is session memorisation, and we treat 58% as the honest figure."*

Three things you must volunteer before being caught:

1. The pilot data came from the **single-USRP passive setup**, not the two-USRP system.
2. Hopping was **measured and rejected** — ~2 Hz, zero usable windows.
3. The per-activity subcarrier rankings on slide 14 **do not transfer**; the deployable set is SC20–23 and 27–30.

Three things never to say:

1. "Multi-channel improved our accuracy." — **not measured.**
2. "We use amplitude and phase." — **amplitude only.**
3. "Our model is lightweight." — **it is a pretrained ResNet18.**

If you don't know something: *"I'd have to check the code for that — I don't want to guess a number."* That answer costs you almost nothing. A wrong number costs you the room.

Timing plan (12-minute slot).

Pages	Content	Time
1	Title, framing	0:20
2–4	Use cases, motivation (land point 3)	2:00
5–6	Problem, RO1–RO4 (say RO2 is partly done)	1:00
7	Literature (the SignFi 98→67 line)	1:20
8–9	**System evolution, two-USRP — core**	2:30
10–13	Pipeline, CSI data, architecture	2:00
14	**Subcarrier selection — core**	2:00
15	Active vs passive (compress if long)	0:20

Pages	Content	Time
16–17	Data collection (state provenance; flag Room B)	1:00
18	**Limitations — attach a number to every bullet**	1:30
19–20	References, close with the numbers	0:20

---

*Prepared from `FYP_Sem_7_Review (1).pdf` (20 pages) and the `usrp-projects` repository: `Three_Channel_Experiment/` (transmitter, receiver, `extract_csi_impl.cc`, `ls.cc`, `base.cc`, `DATA_COLLECTION.md`, `THREE_CHANNEL_ANALYSIS.md`), `USRP_lib/` (`CONTEXT.md`, `csi_tools/`, pilot protocol), `analysis/` (`sc_features.py`, `sc_select.py`, `sc_scores.json`, `analyze_pilot.py`, `pilot_stats.json`, `comparative_analysis.tex`, `sc_report.html`), `Action_NoAction_imgfi.ipynb`, and all 20 captures under `data/`. Figures tagged **[M]** were re-measured directly from the raw CSVs during this analysis.*